CRUSHING HYPER-CASUAL GAMES

How to conceive, create, and launch Hyper-casual games to the top of the App Store.





BY TREY SMITH

About the Author



Trey Smith is the founder and CEO of 8cell, Inc, the company behind the software Buildbox, and also a game development studio with more than 65 million downloads.

Originally growing up in Macon, GA, Trey moved to California, and eventually to Silicon Valley, to pursue a game business career. After partnering with Buildbox CTO, Nik Rudenko, the pair had early success in the app store with hit games like Kolo's Journey, Jump Pack and Monster Magic.

Trey then launched Game Academy® to help other game developers finalize and launch their games into the app store. After serving more than seven thousand customers in 100+ countries, the team at 8cell, Inc moved to create the first truly drag-and-drop game builder, called Buildbox.

Using Buildbox, Trey and Nik have expanded their game business with 8 games that have broken the Top 50 of the app store, 6 games signed with top tier hyper-casual publisher Ketchapp, and multiple #1 hit hyper-casual games like The Line Zen and Endless Sky.



PREFACE

INTRODUCTION

Hi, and welcome to my first book on building, creating, and crushing hyper-casual games in the app store.

My name is Trey Smith, and I've been building these games since, well, really, the beginning. How I got started was a total accident, but it's actually a great lesson to kick this book off.

Here's the story:

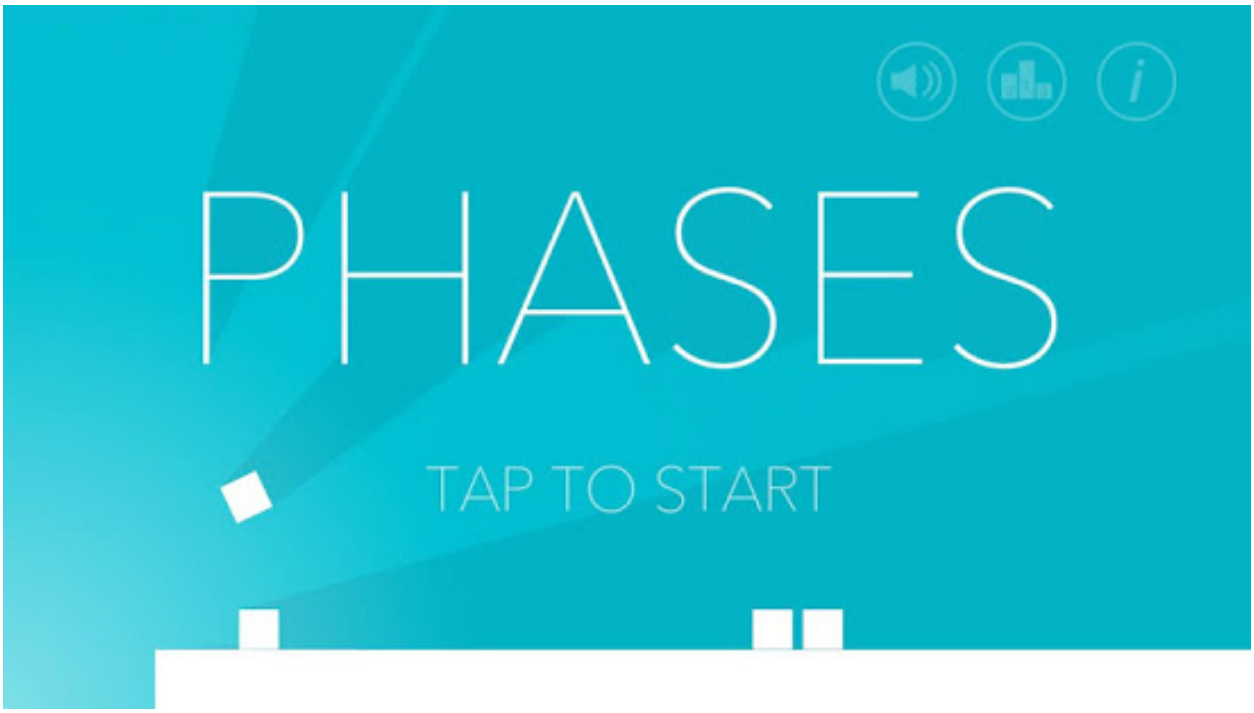
So back in 2014 we were finishing up our drag-and-drop game development software, Buildbox. I started working on a jumping game preset for the software that was similar to Doodle Jump. By accident, I made this preset landscape, instead of portrait mode. In other words, you hold your phone sideways to play it.

Well, this preset ended up turning into Phases (iOS/Android), the first game I ever made completely by myself, and accidentally one of the first hyper-casual games ever created.

How does that happen by accident?

The Accidental Hit Game

When I was working on Phases, a game called The Line was at the top of the app store. You can see the similarities. I actually used a lot more inspiration as you'll learn in this book (games like Electronic Super Joy and Thomas Was Alone), but The Line is where it all *started*.



When Phases was finished, I decided to launch it on my own! I made the game INSANELY hard and I just knew that Apple was going to feature it. I mean, let's be honest, how the hell could they *not* feature it. It was *amazing*. Possibly going to be the best game they had ever seen.

Of course, you probably know what happened next.

They didn't feature it. They didn't message me. They didn't… well.. *care*.

As you can guess, **the game totally bombed**.

So I did what any self respecting grown man does. I immediately crawled into the bed, pulled the sheets over my head, and cried for days, like a *huge, hairy, man-baby*.

Nope, just kidding.

I actually sat down at my computer and thought: "OK, why is this *simple* game called *The Line* number one in the app store, and I can't even get a pocketful of downloads? What are they doing that I am *not*?"

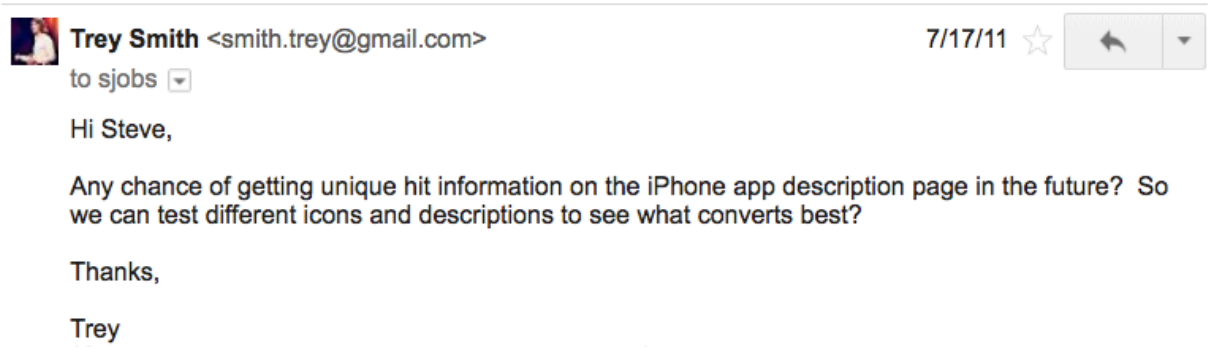
Then it hit me.

How about I email them and *ask them*?

See, if I have any super power in the world, it's this: **I will email ANYONE.**

Seriously, anyone:

That was me attempting to email Steve Jobs. He didn't reply, most people don't, but sometimes - *like the case of The Line* - they *do*.



Trey Smith <smith.trey@gmail.com> 7/17/11

to sjobs

Hi Steve,

Any chance of getting unique hit information on the iPhone app description page in the future? So we can test different icons and descriptions to see what converts best?

Thanks,

Trey

So back to the story.

I'm seeing The Line at the top of the store and decide to email the publishers (some guys who called themselves Ketchup or something… who knows, I had never heard of them at the time) to see if they would release my game.

I figured heck, if they released their own games with success, then maybe they will release mine as well.

They didn't respond.

I didn't like that.

So, I had decided to once again to come up with yet *another* backup plan, when low and behold an email came through.

My god, it was this ketchup company. Oh, they are called *Ketchapp*.



Cute, I like it.

Wow, they're interested in talking, let's see what they say.

The Publishing Deal

At the time I believe Ketchapp had done just one publishing deal. Maybe two, but I was definitely one of the first people reaching out.

They were definitely interested, but said the game was WAY too hard. See, the final version of Phases is hard. It's like real hard, and people actually speed run the game now, but the first version I made was insane.

I had this brilliant idea with the first version of Phases to make it extremely hard. Like, I was literally going for the hardest game ever made. (Seriously, the first name of the game was literally, "The hardest game ever made").

Don't do that. Seems like a good angle, but it's not… it's a bad call.

Ketchapp luckily had this foresight, even though I did not at the time, and asked me if I could make an easier version.

I, *very reluctantly*, said OK.

I thought it was a terrible mistake, but figured what the hell. Maybe we could get some downloads out of this after all.

I made a still *hard-as-hell-but-now-actually-playable* version and Ketchapp liked it. They did comment it was still probably too hard, but decided to give it a go.

It worked out well.

The game broke the Top 25 and built a bit of a cult following. It was one of the first hyper-casual games to really take off, and also sparked a long relationship with Ketchapp (we've signed more contracts with them than any other developer at the time of this writing).

Also, most importantly, it brought us into a new era. We had already had a little success in the app store (about 15 million downloads), but it was a true *struggle*. A constant fight.

The New World

After this, things got easier and growing the business to 65 million downloads and beyond became possible because of the shift to hyper-casual.

As you'll see, this story will be important for two big reasons: First, because it was the obvious shift into hyper-casual, and second, because it was our *first* publishing deal.

We'll discuss this in detail, but with the hyper-casual movement also came the mobile game publishing movement, which was a very big industry change that is still growing strong today.

As you'll learn, this is the secret reason (well, probably not so secret now) that hyper-casual is dominating the app store. It's primarily a handful of publishers who are constantly pushing the games to the top.

It's a pretty interesting ride, and I'll tell you all about it during the next few chapters.

The Book Layout

As you can see, this book is going to be a little different. I'm writing it very conversationally in hopes that it's not boring while at the same time slamming you with a ton of content.

I'll be going over everything I've learned the past 4 years making hit hyper-casual games (and getting them signed to the biggest publishers). I'm also going to show you how to go about it as an indie, and explain every option available to you with my personal recommendations in today's market.

We'll do this by breaking this book down into 4 primary sections:

1. The Idea
2. The Build
3. The Game
4. The Launch

You'll also see a bonus section at the end for advanced hyper-casual training.

Let's get started.

Trey Smith

SECTION 1

THE IDEA

“It’s worth it, let me work it. I put my thing down,

flip it and reverse it”

Before you actually can start thinking of the game idea you want to make, you first - not dissimilar to Missy Elliot - have to *reverse it*.

What does that mean for apps?

Well, I’m going to go ahead and say it now. Let’s get this part *out of the way*. Because you’re going to get REAL sick of hearing me say two things:

1. Reverse engineer
2. Model and improve

That’s OK though. They’re super important. I hope you get sick of hearing those things, because then it will be drilled into your head.

These two phrases, *and these alone*, have been the direct reason I’ve seen success starting businesses in everything from tv software to consulting to video games.

We’ll get to the second one later, but for now, let’s focus on reverse engineering.

Building Your Idea Backwards

The idea behind reverse engineering is easy:

Start at the *end*, and work your way backwards to your origin point.

In other words, figure out *what* you want the end result to be, and then just go and build that.

By doing this one simple thing, you will have bypassed 99% of the other people competing in your space (regardless if it's selling video games or ice cream cones).

The unfortunate truth is most people just don't have a clue what they're doing. They're completely aimless, and not unlike Rambo, they're holding the machine gun at hip level and just spraying bullets to take down the competition.

It might work in Hollywood, but in business? Eh, not so much.

In this idea section, we're going to talk about a few different ways you can come up with ideas, but it's important to know exactly what your END RESULT is going to be before we start choosing the best method for you.

For example, do you want to win a game jam? Is that your end result?

Then guess what, you're going to be studying game ideas by looking at games that won game jams.

Want to be featured on Steam in the indie section?

Then you're going to be researching game ideas by looking at the top indie games on Steam.

Want to be the #1 hyper-casual game on mobile that's making over a million dollars a month (yes, they actually make that much… it's crazy these days).

Then yes, you guessed it: You're going to want to study hyper-casual games at the top of the app store.

Set your sight on where you want to go, and that's the first step of getting there. If you set your sights on the ground, you will *absolutely* end up there.

Now, instead of walking you through reverse engineering game jam winners for game ideas, I'm going to cut through the B.S.

This isn't a book on game jams or Steam, it's about hyper-casual, and in

order to reverse engineer hyper casual games you need to see what's working at the top of the charts.

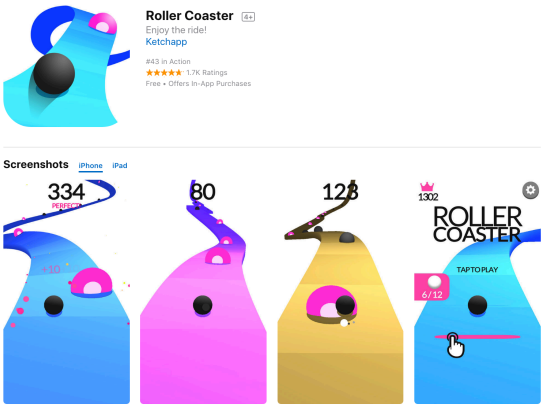
Coming Up With Hyper-Casual Game Ideas

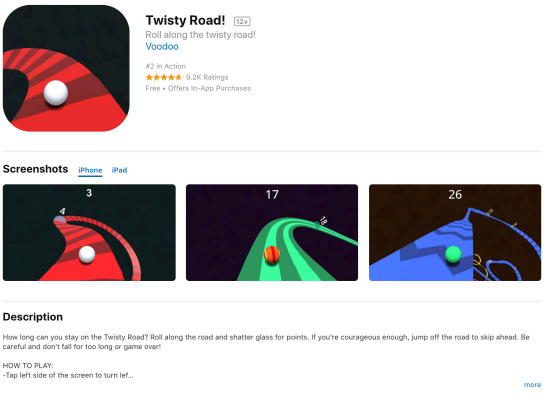
Everyone *complicates* this process.

Make sure you do not, because it's really not hard. Game Ideas should be simple, concise, and based on something we know works.

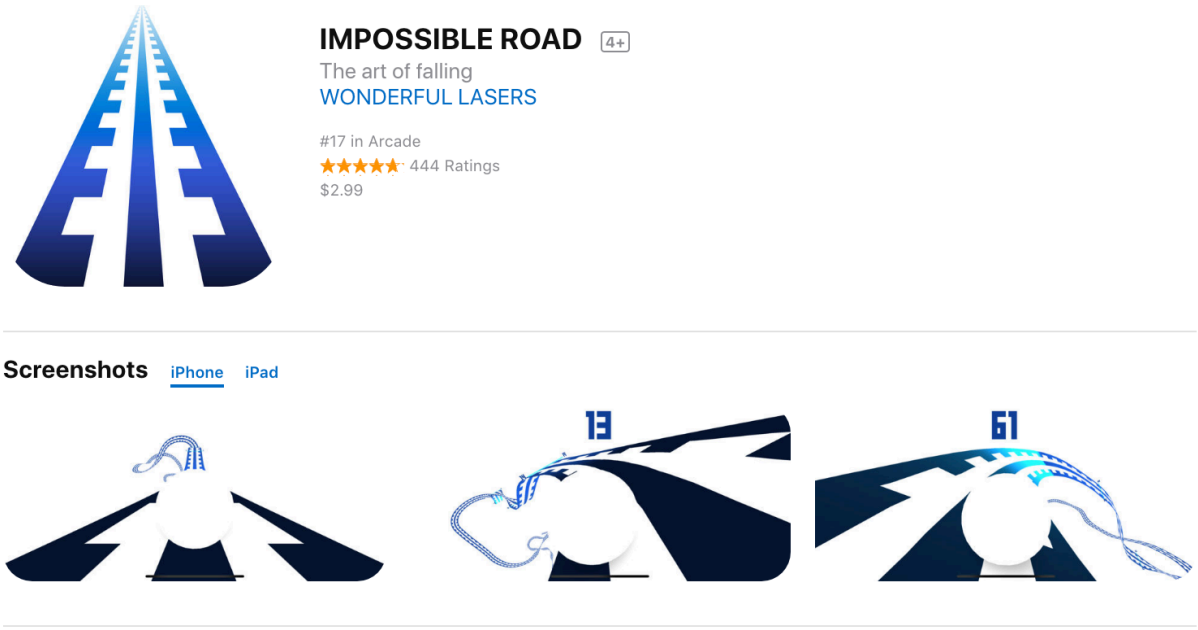
Look at hyper-casual games at the top of the app store. *Rarely* are they really unique.

Heck, usually they are near clones of previously successful games:





Both of these games crushed it at the top of the app store, and let's be honest… they look mighty familiar:



Does that mean you should go *rip-off* people?

No, I would never recommend that and if you look at our games we've decided *not* to take that approach.

I love the artistry and creativity of video games.

We're very passionate about that, and if it became just about the money, then the whole process becomes soulless quickly.

And then, what's really the point? Might as well go work on Wall Street.

Cloning might work for others, but we've decided instead to build new games *based* on current successes. We get ideas from popular games, but still make our final results unique.

Where The Ideas Come From

Most of the top hyper-casual games are created by a developer like you and me, but launched by a hyper-casual publisher like Ketchapp, Voodoo, TastyPill, Fortafy, Appsolute, Umbrella, Nanovation, and more.

The publishers end up being the tastemakers for the app store.

They *create the trends* by signing the games and pushing them to the top of the store.

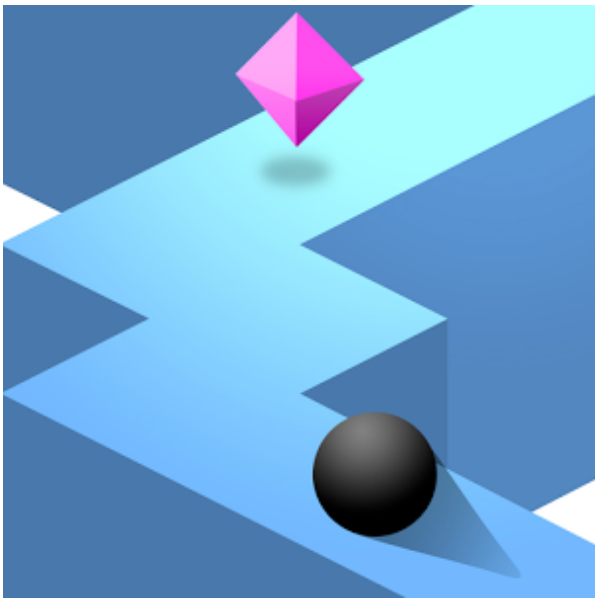
As you'll soon see, we're a big fan of publishing, but even if you decided you absolutely did not want to get published you should still investigate the hyper-casual publishers for game ideas.

Why?

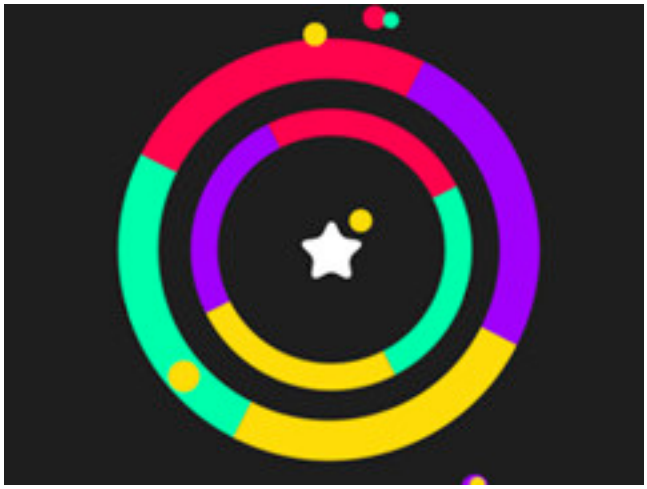
Because they're usually ahead of the curve on trends. What you see publishers signing and pushing to the top of the app store is usually a strong telling of what is to come.

Here are some examples:

- **Ketchapp** signed the game Zig Zag and pushed forth an absolutely enormous push of hyper-casual isometric games that's still trending 3 years later.



- **Fortafy Games** published Color Switch which received more than 150 Million downloads and spawned a huge trend of changing colors in games.



- **Miniclip** released Agar.io which went on to spawn a huge *.io* movement with Slither.io, Diep.io, Arrrow.io, and Voodoo's first big publishing hit: paper.io.

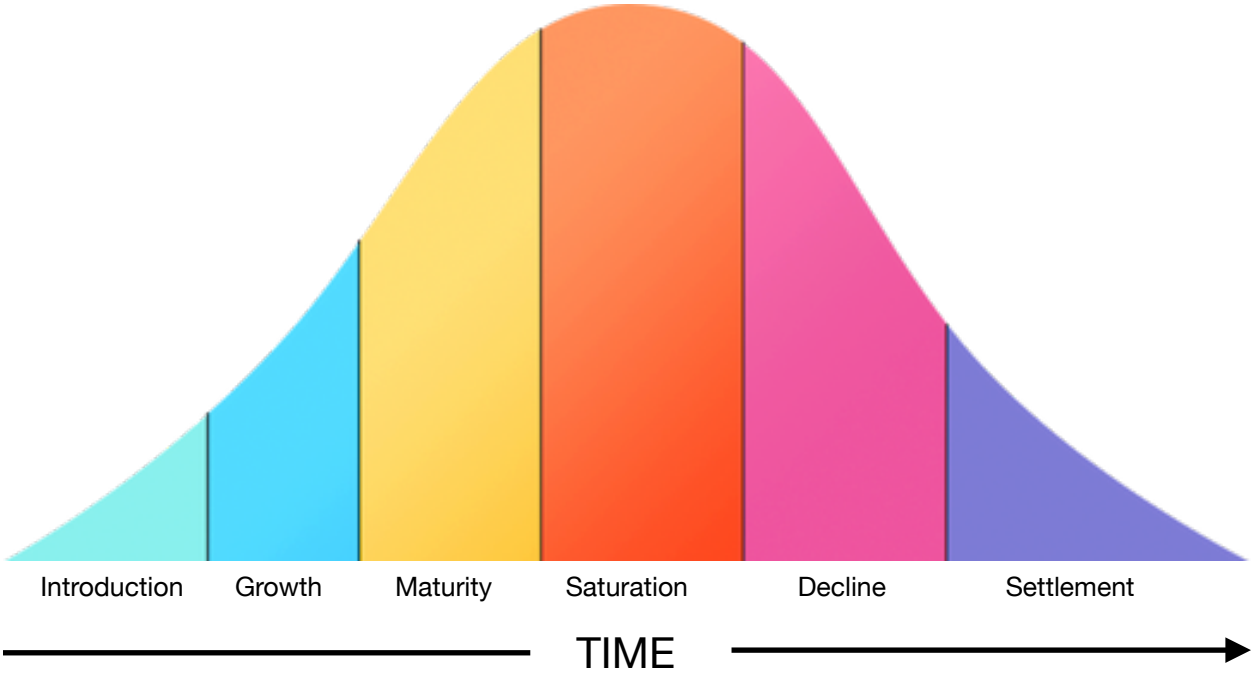


The publishers create the trends. Even if you don't want to go with a publisher you should examine their games closely. The last thing you want to do is create a game idea that at the end of a bell curve.

The Trend Bell Curve

This is one that took me a while to figure out. You absolutely have to make the right game at the right time if you want to crush the app store.

The way this works, is the trend goes up, then down, similar to a bell:



Ideally, you want to make a game when it's in the "growth" stage of the trend. That's the perfect part, right in the middle as it's getting to maturity.

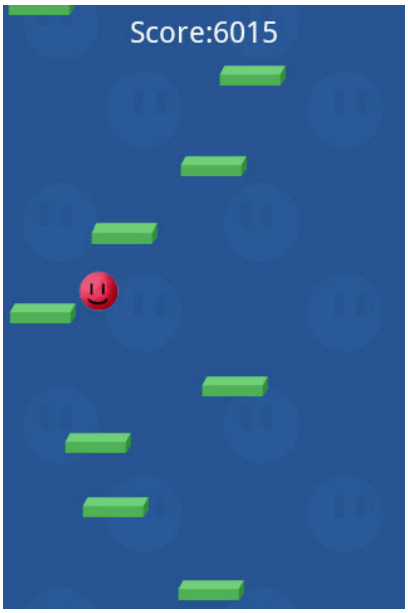
You can still do OK by nailing games in the maturity and saturation phases too.

A Tale of 3 Games

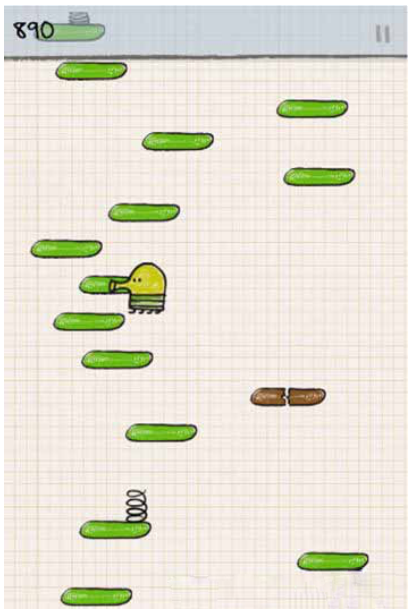
Here's a great example of trend bell curves in action:

Back in 2011, I made a game called *Jump Pack*. It was a jumping game like Doodle Jump.

The game actually did OK. Not too bad, but I was *dangerously close* of missing the bell curve. Also, there was another game that came before Doodle Jump, called Papi Jump.



Papi Jump


Doodle Jump

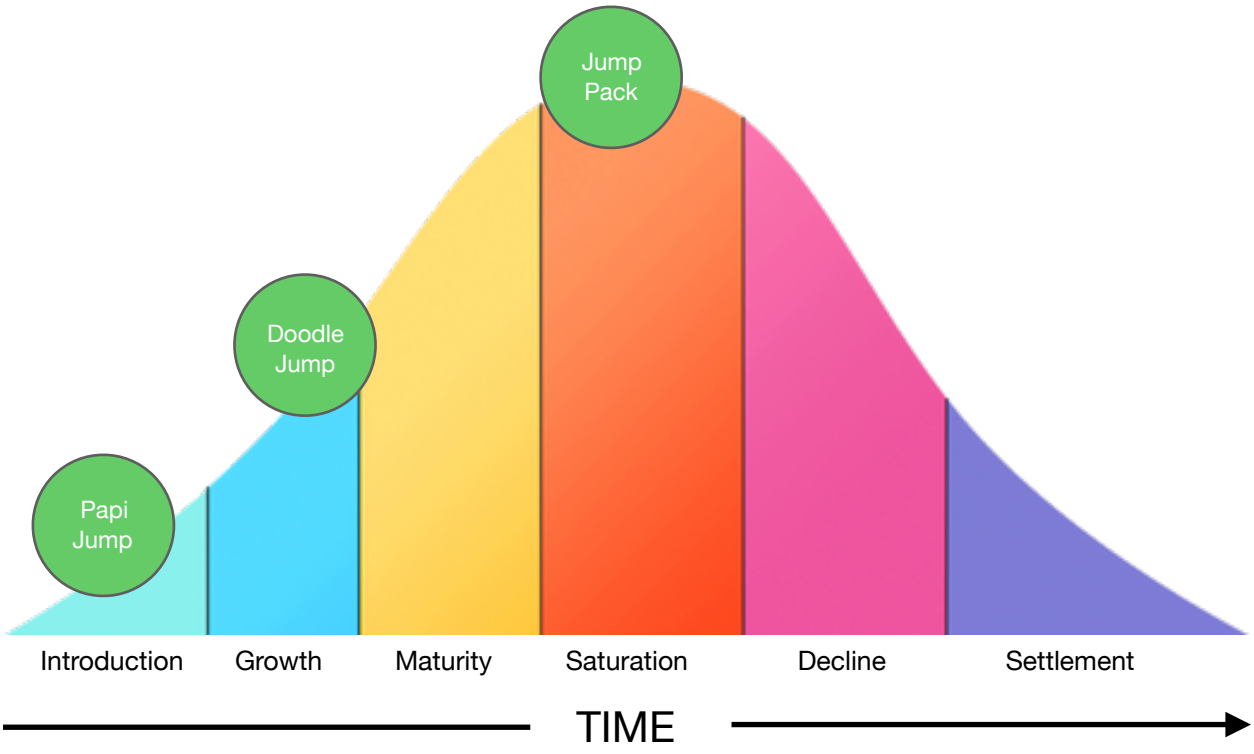


Jump Pack

Papi jump did fairly well, but it was a little too early for it's time. Doodle Jump went on to make millions. One of the first breakthrough apps in the store. Jump Pack was our second game, and I had no clue what I was

doing really. It was definitely late to the party, but luckily for me it wasn't *too* late (the game still made about $120,000 total).

By charting these three game's releases on the trend bell curve, we can see *exactly* why each them had the level of success they did:



Now, you might be looking at this chart and thinking, "Wow, Jump Pack still made $120,000 launching during saturation?".

And that would be a smart question to ask.

How late *is* too late?

I think the answer is right in the *top-middle of that chart*. If you can launch before the mid-saturation point, then you can still do pretty good.

After that, it's going to be tough.

There were a lot of jumping games that came out after Jump Pack, and they did not get near the traction we did. At the time, I was hoping Jump Pack would be the next Doodle Jump, but now looking back I realize we were lucky to catch the wave before it ended completely.

Trends are important, so you really want to make sure you're identifying trends in early stages when coming up with ideas.

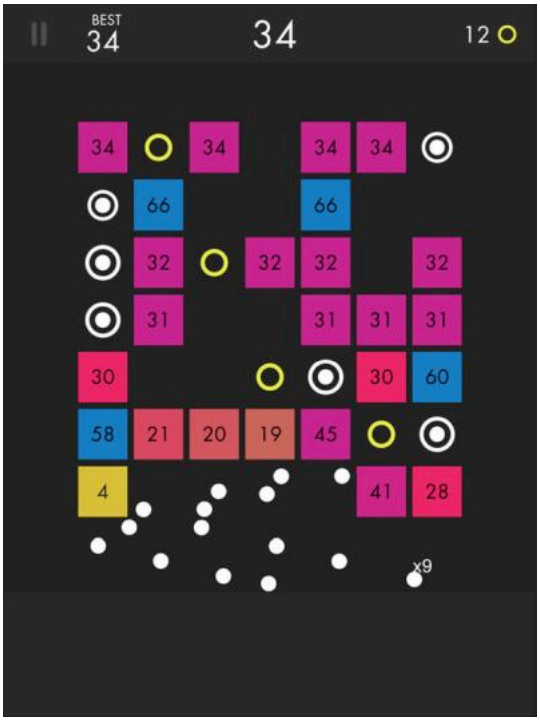
The Inner Detective

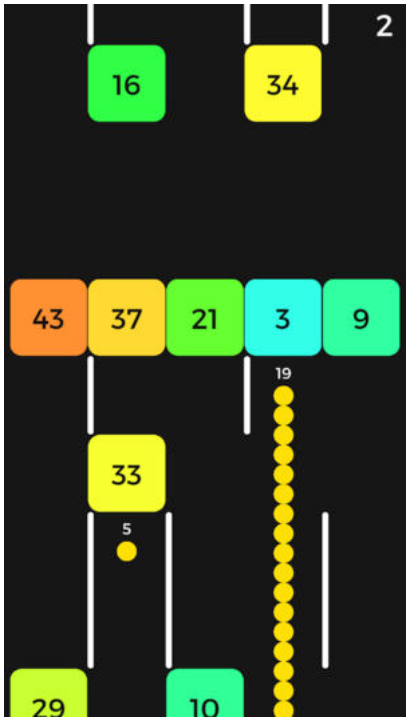
Now we're starting to narrow down the process here.

First, you want to look at top tier publishers, and top hyper-casual games in the app store in general.

Second, you want to take notice of trends that are hot and new. For example, if there is a new game that rockets up to #1 quickly and seems to be hanging out there… that's worth paying close attention to.

The last great example like this was Ketchapp's Ballz. This game shook the app store and flew to the top, and spawned many similar games including multiple sequels and Voodoo's immensely popular Snake VS Blocks.





Finally, you want want to start documenting all of this research, so you can turn it into a few different solid game ideas.

Research, Document, Destroy

I recommend researching the top apps on a near daily basis. I mean, it's not like this type of research is painful.

It's literally playing video games ;)

The idea here is to document the successful games and make note of their trends in gameplay, graphics and features.

You can do this however you like, but I am a huge fan of spreadsheets.

Here's is how mine looks to keep up with games in the app store:

Game Name	Publisher	Link	Gameplay	Graphics	Features
Minivan Dan	Buildbox	http://www.a	Runner	Isometric	Humorous
Triple Race	Hurricanez	http://www.a	Versus Racing	Isometric	Dark Background
Shark Roll	Temptest X	http://www.a	Runner	Minimal Flat	Destructive
Blacklist	Conduit	http://www.a	Puzzle	Cyberpunk	Dark Background

After a while you'll start seeing similarities, and these similarities can turn into trends.

In the example above, you might start noticing there are multiple games that are runner gameplays, isometric graphics, and feature dark backgrounds.

Does that mean a dark isometric runner will just absolutely, no questions asked, crush the top of the app store and bring all other hyper-casual games to their knees?

No, but at least it should have a *shot* at the top. (For the record I made all of those game names up on the excel spreadsheet. This is not actual advice on what you should work on.)

The truth is, a lot of this is going to be a *guessing game*. There is definitely luck involved here, no question about that.

The trend bell curve will never be fully known until it's hit the settlement point for a while, but you can get better and better at gauging trends.

For example, about a year ago we saw a new trend with the game Ballz by Ketchapp. This game had true progression (i.e. your character evolves over time as the world around him gets more difficult) that had not been seen much in hyper casual.

This is a trend that might not ever die out, or it moves so slow it appears to not to be dying out.

The “destroy” part of Research, Document and Destroy is also an important, yet often overlooked, part of the process.

When you see a game starting to phase out, then remove it from your list. This doesn’t have to be as soon as it’s out of the top 10, but when you feel like a game has peaked and is falling off into obscurity, then it’s time for the game to go.

When you start documenting these games, you’ll notice some stay longer than others.

I’m not talking about the obvious ones like Minecraft and Angry Birds being popular for a long time. I mean you will literally see a game go from Top 5 and magically drop out of the Top 500 in almost an instant.

Be very wary when this happens. It means that something else could have been pushing this game to the top. A huge ad campaign, someone buy shady installs, or some flash in the pan meme that died seconds after it was created (like *Cash Me Outside* and *Damn Daniel*).

All of these scenarios will create an app that rockets to the top of the app store without any real trend power. They are flukes, and not something you should really pay attention to (at least, from a gameplay, graphics and features perspective.)

Model and Improve: Gameplay, Graphics and Features

Once you have a list you're happy with, it's time to nail down your game. And this part really isn't that difficult.

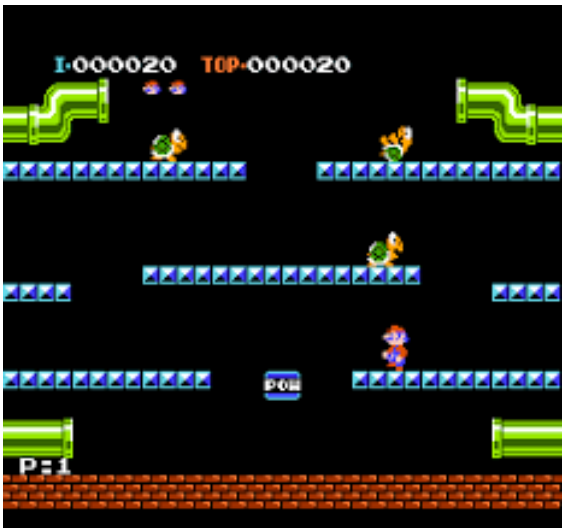
Your game is going to consist of just three things: Gameplay, graphics and features. You want to see what is currently trending, and throw in a bit of individual *panache* here to come up with something unique.

This, in essence, is *Model and Improve*.

Every game you love from Minecraft to Mario Brothers has been HEAVILY based on a previous game

For example...

The original Mario Bros Arcade game debuted a year after Joust was already a hit:



Mario Bros.

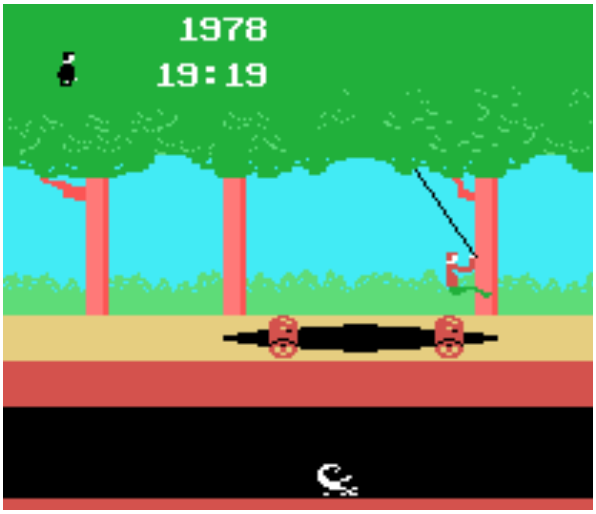


Joust

The Sequel, Super Mario Bros, definitely took some inspiration from Atari's most successful game Pitfall:



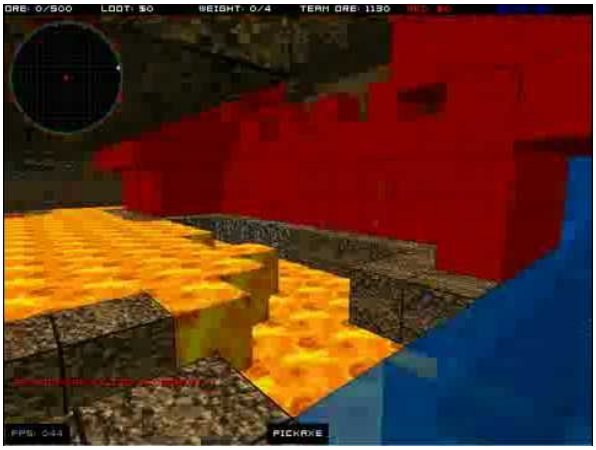
Super Mario Bros



Pitfall

And my all time favorite example... Minecraft creator, Notch, readily admits it was based on Infiniminer:


Minecraft


Infiniminer

So are these games a total rip-off?

No.

That's why you don't see much controversy around these games.

The reason they aren't rip offs is because they didn't just MODEL the original game… They also IMPROVED.

Mario Bros took away flying from Joust. Super Mario Brothers added pipes, secret worlds, multiple enemies, and more to Pitfall.

Minecraft added, well, crafting.

In fact, even the creators of games like Infiniminer state they don't feel their games were ripped off:

> **The act of borrowing ideas is integral to the creative process. There are games that came before Infiniminer and there are games that will come after Minecraft. That's how it works.**
>
> — Zachary Barth, creator of the open-source game "Infiniminer" — the inspiration for Minecraft — who is tired of being asked if he feels ripped off about the millions that Minecraft has made for its creator Markus "Notch" Persson

On Reddit, Zachary Barth went into a little more detail and specifically says why his game could never be as popular as Minecraft:

[–] **fauxmosexual** **18 points** 2 years ago

You created a game which directly inspired Minecraft, *the* breakout indie hit. I don't think it's an exaggeration to say that without your work, Minecraft would never have happened and Notch wouldn't be a literal billionaire right now. Do you have any regrets or resentment about missing that boat? Have you have any kind of relationship with Notch?

permalink source embed save save-RES hide child comments

[–] krispykrem ZACH BARTH, ZACHTRONICS [S] **26 points** 2 years ago

I don't think I missed any metaphorical nautical vessels because I never would have made a game about tree punching and crafting systems; I lacked the genre awareness of games like Wurm Online that Notch had.

Amusingly enough, I've never actually met Notch before! We've interacted on Twitter a little since the sale to Microsoft, but that's basically the limit (he was a big Infinifactory fan). I invited him to the Infiniminer party at GDC a year ago, but since we deliberately scheduled it opposite the Minecraft party he understandably couldn't make it.

permalink source save save-RES parent

Notice he says, “I would have never made a game about tree punching and crafting systems; I lacked the genre awareness of games like Wurm Online that Notch had.”

Interesting... what is *Wurms Online*? It’s actually one of Notch’s first games, and it’s about, you guessed it, crafting:



Notch was modeling Infiniminer, and improving by adding crafting, which he had worked on in a previous game.

This is probably the best example of model and improve you'll ever find in game design (though Angry Birds, Doodle Jump, Clash of Clans and Candy Crush are other great examples for mobile).

This is what you want to do when you get ideas from popular published games, just on a more simple level. An advanced idea like crafting definitely will take a lot of time, effort and money to create.

The Blank of Blank

To give a great example of this mindset of model and improve, let me tell you a story a venture capitalist investor (VC) told me in Silicon Valley.

I was meeting with them to talk about funding for Buildbox, which I did not want, but figured the meeting would be good experience. After he found out I was not really interested, we started talking casually.

He then told me best way to pitch VCs like himself. Unsurprisingly, it was very similar to model and improve.

He said VCs love to see how your product can be compared to a similar product that overtook an industry.

For example, Wordpress took over the web development industry by allowing anyone to make a website, so a good hook for Buildbox could be, “We’re the Wordpress of game development.”

The investor told me they love hearing, “We are the blank of blank.”

As you’ll learn later on, publisher’s aren’t unlike VC’s when it comes to this. They love hearing “It’s like the Minecraft of Space Shooters” or it’s the “Line Zen version of Ballz.”

Model and Improve works very well and publishers know it. While they might *sometimes* publish clones, they actually prefer *not* to when possible

Think about how to *model and improve* off the information you’ve written into your research spreadsheet. Think about the similarities between the games.

Think about what is *currently trending*, and how you can keep the core trend, but add a *unique angle* to the gameplay, graphics or features.

Think about how to mix and match what is *currently working*.

As you’ll see in the next section, we do this with all aspects of your game: graphics, UI, gameplay, game design, and even the flow of the game.



SECTION 2

THE BUILD

Reference, Reference, Reference

Remember how I said you would get sick of hearing me talk about reverse engineering and model and improve?

Well, I got some bad news: We have to add one more to the mix here, because it's just as important.

It's *references*.

I've been lucky enough to work with thousands of game developers over the year. See, before Buildbox, I used to blog about how I was building my game businesses through outsourcing (long story that we'll discuss in the next section).

This blog took off and slowly turned into Game Academy, which was the largest online game business training site. We had over 7,000 customers and I had the opportunity of talking to literally hundreds of people about their game business.



It was fascinating. I saw many people crush it, and of course many who did not. I also started seeing similarities between the people who did well.

Yes, the successful ones *almost* always reverse engineered, and yes, most of the time they also practiced *model and improve*.

But the one thing they definitely all had in common, as in, there is absolutely no question that every single successful company I worked with did this… was using references.

While I don't run Game Academy anymore now that Buildbox has taken over, I still work with many developers and help them get published. Once again, the ones who succeed are *always* the ones who have references.

So what does this mean and why is it so important?

Let me explain.

Let's say you have a game idea. How do you know if it's amazing or it sucks?

Or even better yet…

Let's say you bought some art for your game, but you can't decide if it's good enough to make it to the top of the app store?

Even the best artist and game designers can have issues with this without *references*.

At minimum, you should have a gameplay reference and design reference to constantly look at while building your game. These can be the same game, that is fine, but many times they'll end up being different games.

Both of these references should be holding a strong position on the bell curve, and they should also be something that you can actually *do*. In other words, if you don't have the ability to create in-depth pixel art, nor the money to hire a pixel artist, then a pixel game should not be your graphical reference.

We'll talk a lot about ability in this section, but keep this in mind when choosing a reference.

Why Are References So Important?

As mentioned, I help a lot of people get their games published. This has afforded me to work closely with many people and help them secure deals with publishers like Voodoo, Ketchapp, Nanovation, Gameloft, Appsolute, Umbrella and more.

During this process I've seen it all. I've seen amazing games that I knew would be picked up immediately. On the other side of the spectrum, I've seen horrific games where I was shocked people thought they would be able to get them signed at all.

The horrific ones all had something in common: They created bad games because they had nothing to compare it to.

How do you know your art sucks if you haven't set a benchmark? How do you know if your game is too hard, too simple, the character is too big, too small, or the levels are annoying?

All of this, and more, can be easily solved by having a reference to constantly compare to:

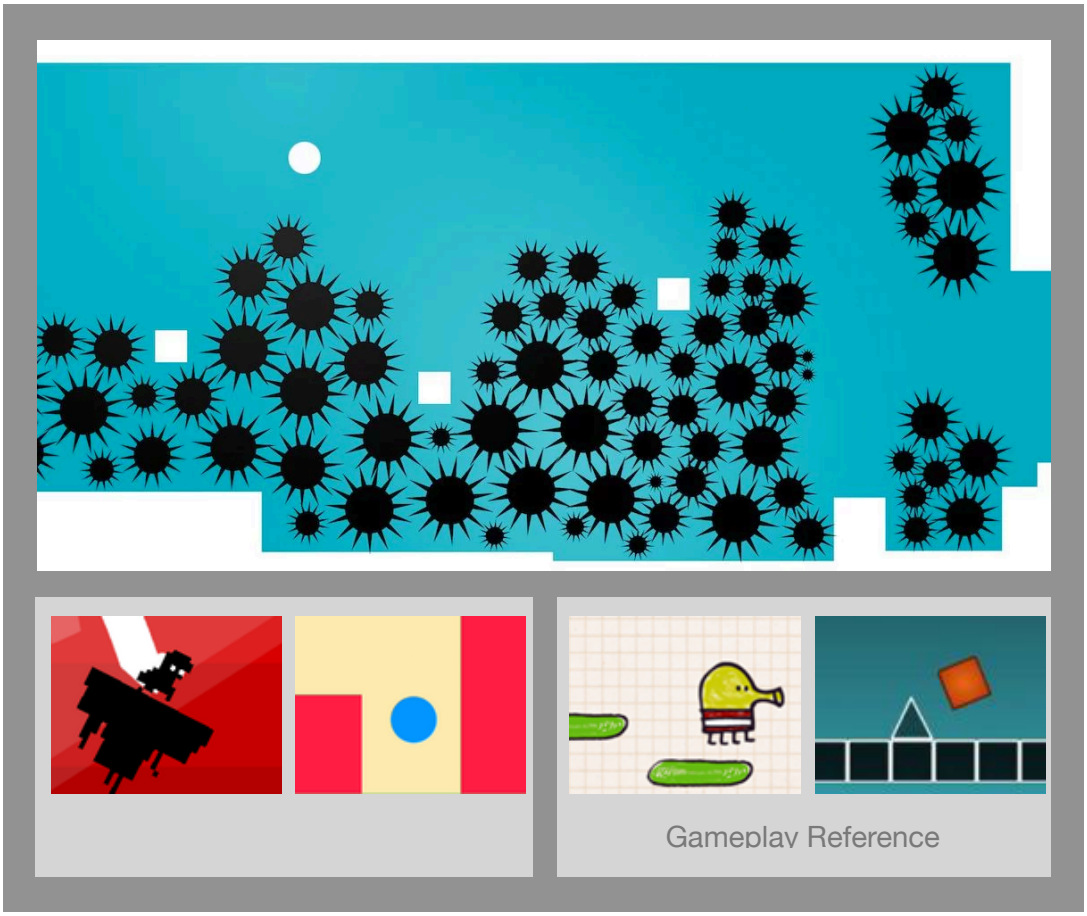
- Not sure if the Main Menu screen is good enough? Compare it to your reference.

- Level 3 might be to difficult, but not totally sure on that? Compare it to the reference.

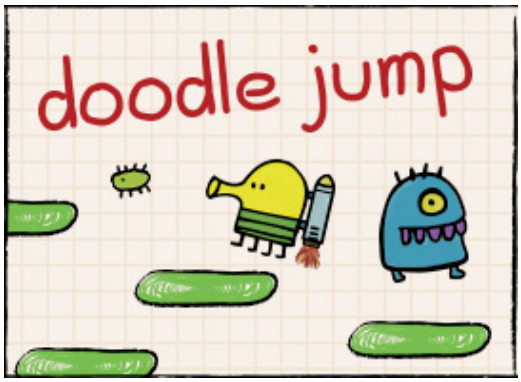
- Don't have anyone to play test your game on, but need to see if it flows well? Compare it to the reference.

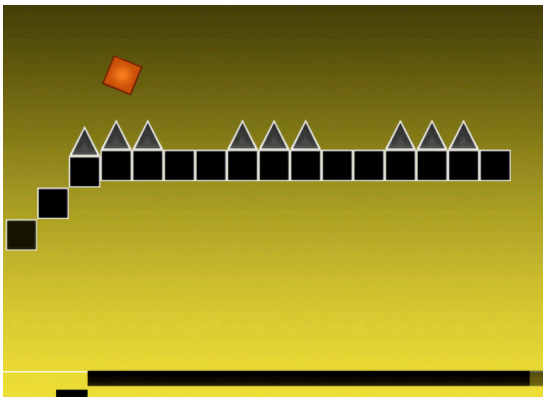
Ideally you want at least one reference for gameplay and one for design. More is fine. You can use a single game if you wish, but I prefer using multiple so I have plenty for comparison.

In my game, Phases, I had 4 different references:

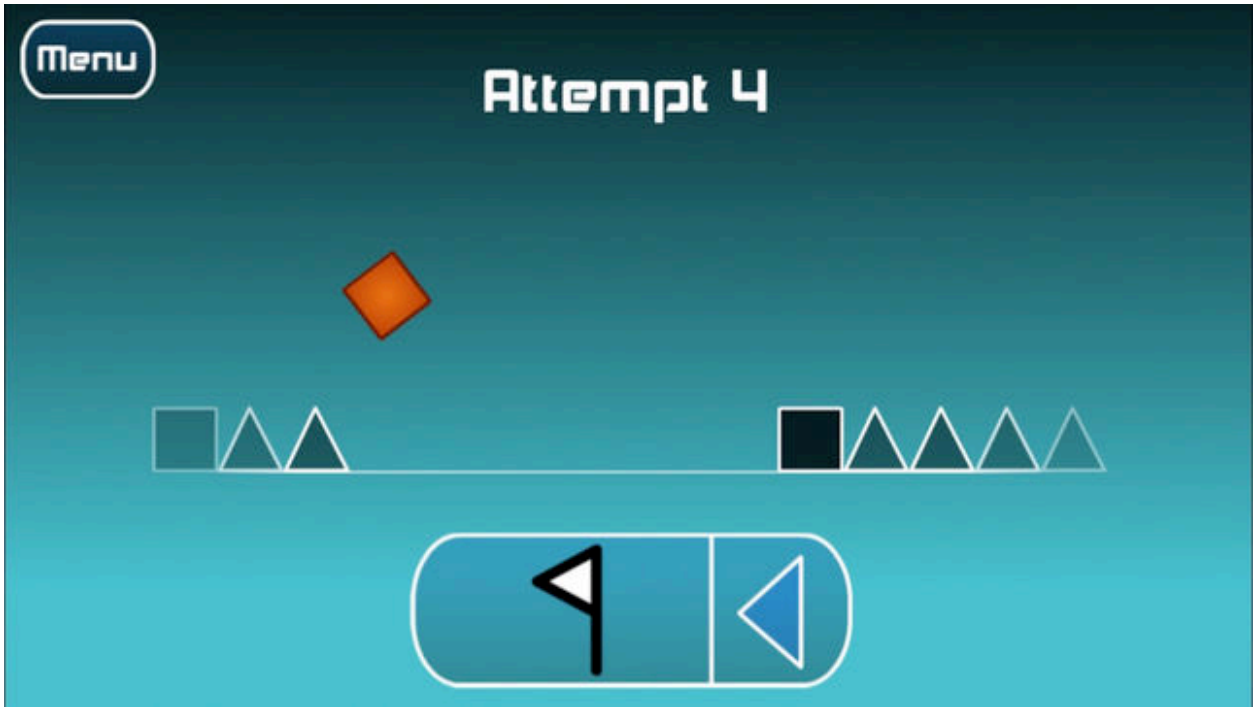


The design references were *Electronic Super Joy* and *The Line*. They just happened to be two very popular games I was playing at that time. The gameplay references were *Doodle Jump* and *The Impossible Game*.





As I mentioned earlier, I actually made the game too hard in the beginning. Ketchapp made me ease it up a bit. The Impossible Game reference really didn't fit hyper-casual, and this is why I would recommend you have all hyper-casual games for your references.



Unfortunately, back when I was making Phases, there was only one hyper-casual game in existence, which was The Line.

As you can see, I used minimal graphics and casual gameplay to to have references, even if I didn't nail it 100% (The Impossible Game is not truly casual to most people).

Before creating your game, make sure to pick at least two references. You'll constantly compare your game to these references when questioning the look, feel and direction.

How To Build Your Game

There are many ways to create a game, but they are all broken down into 3 different categories: coding, hiring and using builders.

For hyper-casual it's a no brainer. The first two options are extremely costly of either time or money, and most hyper casual games are built with game builders (especially, Buildbox, our game builder).

It should be no surprise there what we'll recommend. We literally built the software to create and cater to this hyper-casual market, but regardless, I want to take a look at all three options so you understand every aspect.

We'll start with the most difficult first:

Creation Option #1: Coding

All games are created with code, so it might seem that learning programming is the most logical step to take if you are starting a new business.

This is actually the last option we recommend for people new to the business.

Why? Well, it's probably not what you might think.

Most people guess it's because it's hard to learn and simply takes too long. Yes, this can be a major disadvantage. If you don't know how to code and need to learn, you are looking at a 10X longer creation time frame.

The reason that's not our *number one* reason not to code your own game is because it has a very obviously wonderful trade-off… you get the game made for *free.*

That's something to consider.

You also get 100% pure creative control over your game.

If you hire someone, you will have lots of back and forth and possibly feel at times the coder just doesn't *get* your vision. Maybe the game will play too slow, too fast or just not *feel* right.

Having total control seems like it is good, but there is an ugly cause and effect from this equation: The quest for *ultimate control by doing it all yourself* (aka micromanaging) is exactly what *kills* most games before they finish.

One thing I'm fascinated by is not just games, but the business surrounding it.

To me, business is the ultimate game.

It's not dissimilar to *Starcraft* or any other tactical strategy game. You have to gather resources, build things that create more resources, and constantly battle the competition.

It's exciting; thrilling even, but there is a good reason why Generals of the business army don't fight on the front lines.

You can't see the big picture when you're inside the canvas.

Most successful studios don't have the head of the company coding their games. Instead the heads are managing the process, getting the results, and looking at the big picture.

And honestly, it makes total sense.

How many good games were never seen because they weren't launched successfully? How many amazing games, developed by great people with dedication and care, never even got a single review?

Most likely tens of thousands if not hundreds of thousands. The problem is, making a good game is a small piece of the successful game business equation.

Imagine if one of those great undiscovered game founders spent *half of their time* thinking about the launch. What if they were reading reports about building their business (like this) instead of just focusing 100% on the code.

Think they would have different results?

Now, it should be noted, if you already know how to code, then this might be a great starting option for you though, but we still recommend outsourcing this step as soon as you are financially able for all the reasons listed above.

As the person running the company, it can vastly help if you are not completely tied up *inside* of your game.

Creation Option #2: Hiring

Most of the apps and games in the app store were created by hiring other people to code them.

Sometimes outsourcing, sometimes employees.

Interestingly, this has not been the case with hyper-casual games. Most hyper-casual games are being created with our third creation option, which we'll get to in a second.

While hiring people is no longer our recommended source, we will still cover it in detail in this document. Fun fact: This used to be actually be our number one source for making games, but then the prices doubled and the talent became picked over.

Things got tough, fast, and a lot of people left the industry around 2014 because of it.

Regardless, knowing how to hire people is a great skill. It's a skill that definitely can help your business in the long run. Once your game business grows, you'll need assistants, managers and people to help you build games… even if you *are* using game builders.

The Hiring System

There are multiple ways you can hire people to create games for you. Some of the most popular are:

- Outsource Sites (Upwork, Guru, Freelancer, etc)

- Local (Universities, Agencies, Friends, etc)

- Employees (LinkedIn, Craigslist, Monster, etc)

Assuming you don't have a friend who's a great coder and owes you a favor, the most cost effective way to hire someone is through *outsourcing*.

We've used nearly every outsource site available and while most are competitive, we do have a list of favorites.

At Buildbox.com, I've used Upwork more than any other outsource site, but I've dabbled with all of them. I even hired graphic designers from showcase sites like Dribbble before.

Once you decide on a site (or multiple sites), you'll need to post an ad and interview applicants. We're going to walk you through this process, but first let's discuss how to pay your outsourcer.

There are two options:

1. Hourly
2. Full Pay

While both options have their place, we've found it's much better to focus on full pay instead of hourly.

With a full pay project, we pay half when they have created a strong base version of what we've requested and then the rest upon project completion.

This base version will depend on what your final goal is, but typically it's something that should finished in the halfway point of your project.

If you're creating a full completed game, then this would be the prototype phase. Once you have a working version you've tested and played (possibly even with stand-in graphics), then it would be a great time to pay them half of the total cost.

While hourly might seem nice at first, the problem is actually keeping tabs on the programmer.

Most of these sites provide tracking services, but they're not foolproof. We've found many programmers running up the clock by using two computers. One to "work" on and the other to watch youtube videos. With full pay this is not an option.

The first step to hiring someone, regardless where you hire them, is writing an ad. While the advice below will work for any site, our favorite is currently Upwork for hiring outsourcers.

Writing Your Ad

Once you're ready, you'll need to create an ad. Here are some tips to help you craft ads that will attract the right people:

1. **Write ads quickly** - You're about to see a very common thread in all of this: Don't get bogged down and move quickly. This starts with the ad. Keep your ads short and sweet. We rarely discuss in-depth project details and put a focus on the fact that the project is unique and discuss the core details. Try to write your ad in one pass and takes about 5-6 minutes. Review and edit if needed.

2. **Don't search for people, wait for them to come to you** - This reason is simple… people that respond to your ad are hungry for work and interested in what you are doing. Random people you contact probably are not.

3. **Post multiple ads** - Post on multiple sites and post multiple times. Sometimes you'll need to post 10 ads before hiring someone. Most of the time we talk to someone with problems hiring, they've posted just a few ads. Think about it this way. If every time you hire someone you go through 200 applicants and someone else goes through 20, who do you think will end up with the best guy?

4. **Always go with your gut** - If someone doesn't reply quick enough, gives a strange answer or just doesn't make you feel comfortable,

then don't hire them. If you automatically just "click" with someone, then you might want to give them a shot. Your gut is extremely important with the hiring decision.

5. **Move quickly though applicants -** It's better to scan 200 people quickly than review 20 people slowly. Don't get bogged down on each one. One tip is to ask a few questions, if all questions aren't answered then skip immediately. If they don't take the time to apply correctly, they most likely will not benefit your business.

You'll want to post these ads in as many relevant categories as possible. If there is an option to post in game development and mobile development, then post in both. Your main goal here is to cast a wide net so you'll have many applicants to sift through at a fast pace.

By the time you're done with this section, you'll be able to take a closer look at a handful of perfect candidates.

Interviewing Candidates

Before you interview an applicant, you should have a few things in place:

1. Your basic design document describing the various sections of your game.

2. An idea on what your budget is for this game (expect to pay $2,000 minimum for a simple game and $5,000 minimum for a more complex game).

3. A list of questions which you will discuss.

4. Bonus: Have a screen recording of a powerpoint presentation describing your game idea. This isn't required, but can be helpful and make the interview process smoother. Jing is a great free screen recording platform you can use.

We choose to interview applicants on Skype, but you can use any chat based software that is publicly available. We also recommend sticking with text chat instead of voice chat, as voice chat can be awkward for both parties, especially if the native language is different.

Here's a few tips for this interview process:

1. **If they don't respond quickly during an interview, close the interview immediately** - This happens more often than you would think. Don't wait around for 3 minutes every time they are responding. If that happens I just tell them it's not working out and move on. Remember, this interview is the first impression you'll see of their communication style. Communication is extremely important when working via Skype.

2. **If they are sarcastic, close the interview immediately** - This one is rather baffling, but does happen. Sometimes, more often than you might think, we have situations where the programmer has a poor attitude and is sarcastic or demeaning. This advice is very important for the people who might not be technical. It is 100% OK if you are not overly technical. Just like most people in a leadership position, you're job is to hire someone smarter than you in certain areas and explain things to you when needed. You're job is to manage them, not to know more than them.

3. **Dive deep into their answers** - This is a very important one. Often times we have people say they are confused on what they should ask their applicants during an interview. The truth is, you could easily ask just one or two questions and fill up a 30 minute interview by focusing on this one point of diving deep.

   The premise is simple, once you ask them a question you dive deep into the answer. If you ask them "What was your best project to date?" then don't answer with "Ok great." Instead, answer with "That's interesting, how long did it take?"When they tell you how long it took, dive deep into that answer as well with something like "Interesting, what took the longest? Did you feel it could have been shorter?"

   Like inception, you keep going deeper into their answers until you've hit the bottom and there is no where else to go. Then move on to

your next question. You'll not only learn more about the skill of the candidates, you'll learn more about them as a person. You'll suddenly start understanding how they work and find out if you're truly compatible.

4. **Play their previous games** - Extremely important, very simple, yet often overlooked. Before you interview someone, test out their previous games. If you're hiring a programmer, don't worry about the art. Focus on how buggy the game is. See if it moves quickly or feels sluggish. If the game is not good, then cancel the interview.

5. **Only hire people that you bond with** - There is more to a successful working relationship than skill, especially with games. Games are art and for you to create something amazing you have to be on the same page as your programmer. If you don't feel like this is someone you would want to spend an enormous amount of time with, then don't hire them.

6. **If you are unsure of either price or their ability, offer a small portion of the project** - This is for the times you are on the fence and don't have any strong gut feelings about the applicant… and you haven't found anyone else. When this combination of events happens, then you might want to offer a small portion of your game for a very cheap price to see how well you work together. Maybe just have them create the basic user interface structure and go from there.

This section alone could cover this whole manual. You can find a large number of books not only on outsourcing, but also team creation and hiring. It's truly a fun and essential way to scale your business, but make no mistake, it's a daunting task.

One way to ease up the process is purchasing base game source codes. For example, if you were going to make a jumping game inspired by the base gameplay of Doodle Jump, then you could purchase a code that already has the base gameplay. This could be your "prototype" that you were going to have someone else make, and then you could hire a programmer to finish out this game and add your changes.

Apart from coding a game yourself or hiring a developer, there is a final option for game creation that is not only the most cost effective, but also the most fun. Of course, we are probably biased as we're heavily invested in this area.

Enter the game builders…

Creation Option #3: Game Making Software

If you want to build a game, but don't want to manage people or code it yourself, then your third option is to build it with software. Obviously we are biased here (we've dedicated our lives to creating one of these tools specifically for hyper-casual), but we truly believe this is the ultimate solution.

Also, we're not the only ones. We've dominated this space for good reason. It's extremely easy to use, it makes great chart-topping games, and it gives you full control without stealing your time.

As a bonus, it ends up being by far your cheapest option as well ;)

The game development software we created is called Buildbox.
It's different than anything else out there for a few reasons:



1. **It's the most rapid development tool available**. Games like *Color Switch* and *The Line Zen* (which were both made with Buildbox) were created in a single week. Because of this, it makes it the absolute cheapest way to make your own games (besides arguably coding it yourself, but even that ends up being more expensive if you include the cost of your time).

2. **It's the only *truly* codeless development software for making games**. There is no coding, scripting or programming required to use

every feature in the software. If you know how to use powerpoint, then you can use Buildbox.

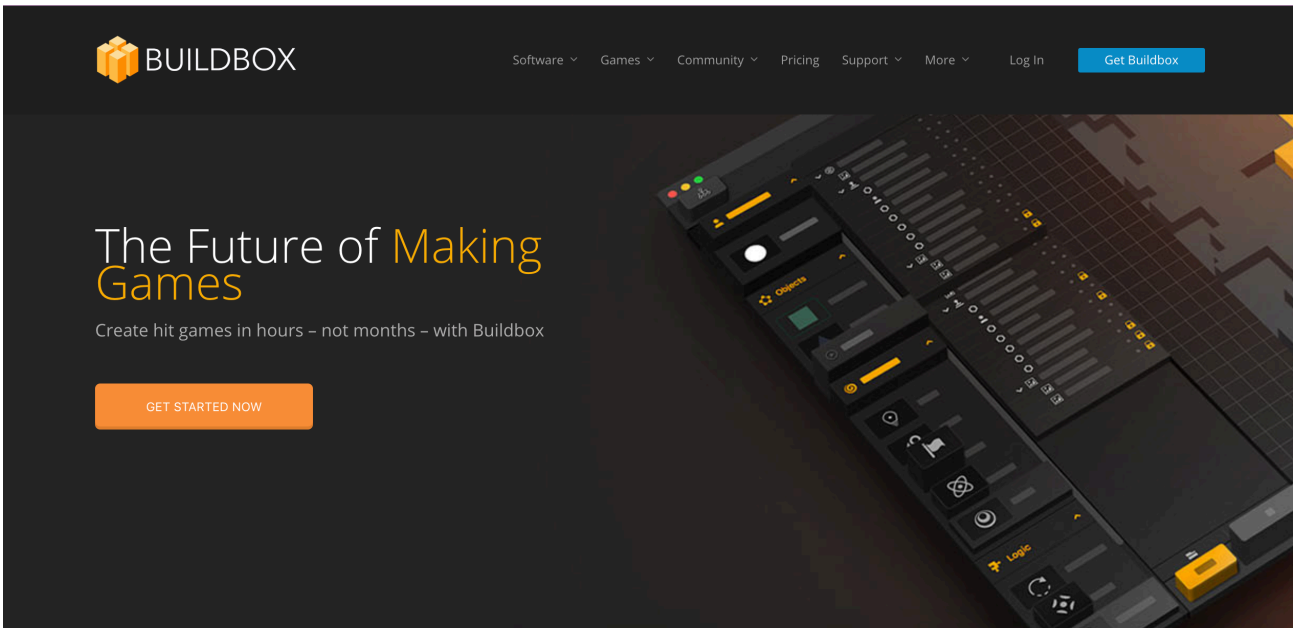
3. **Our new 3D version, Buildbox 3.0, doesn't require any game art.** With this new version it's the first time you can build hyper-casual games without having to create or purchase art. We have it all built into the software.

4. **It's the go to tool for hyper-casual**. More hyper-casual games are made with our software than any other way. Since we've released, we've had more than 150 hit games made with Buildbox that were either featured by Apple, broke the top 100 of the App Store, or were picked up by a top tier publisher.



There are many game makers that rely on scripting and mild coding, but we created the first and only software that truly requires zero programming skills. It's not for everyone, but the people who it is for tend to like it a lot.

We don't want to spend too much time talking about our own products in this guide, but if you are interested in learning more about Buildbox then visit us at Buildbox.com.



Game Art & Sound

After you've picked a creation method, you'll need to acquire art and sound for your game. Like creating the code of your game, there are various options that ride a similar creation scale.

You'll either be spending more time, more money, or something in between.

In a typical situation, you'll choose one of the following methods to get art and sound:

- Make it yourself

- Buy pre-made assets

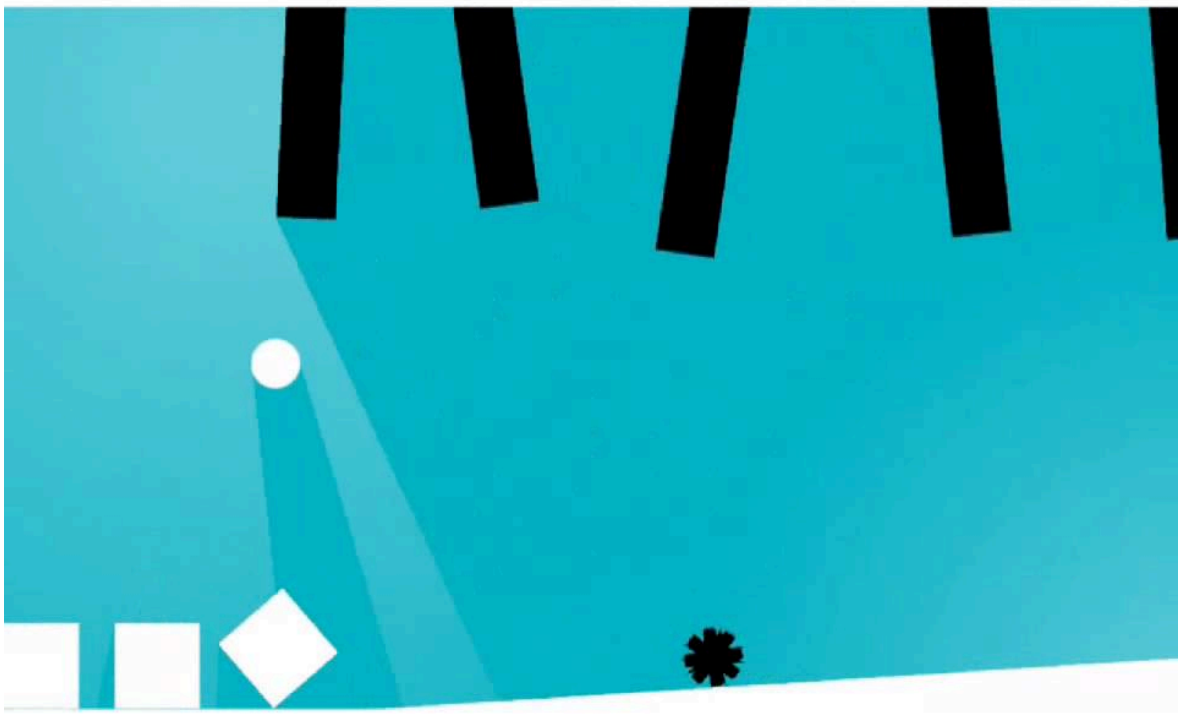
- Outsource an artist and sound designer

Here at Buildbox.com, we've tried all of these methods at various times and now we'll quickly guide you through the best practices for each.

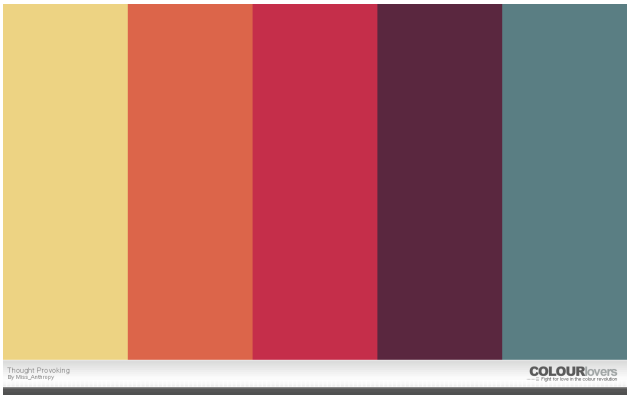
Making Art & Sound

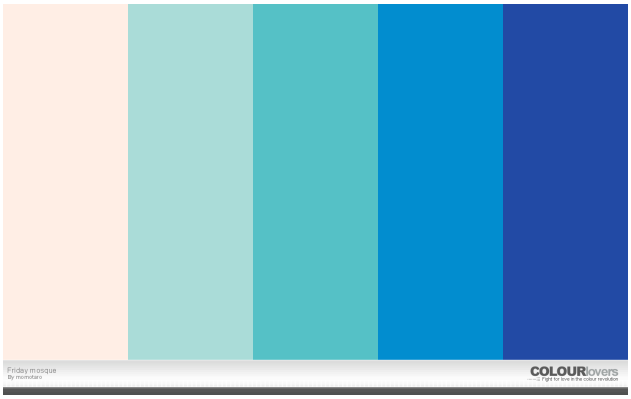
If you choose this route, then you'll want to either keep things extremely simple if you're a newbie.

This is what I did with my game Phases.



I’m not a professional designer, so I kept the game extremely minimal and stuck with a nice bright color palette. If you decide to go with a minimal art style, then check out palettes from sites like Colour Lovers or Adobe Color.





For sound, you might be surprised what you can record just with a simple mic and some ingenuity.

For instance, the ball bouncing in our game Bounce (made with Buildbox customer, Mike Davidson) was a sample of me thumping my mouth. The breaking noise of the green lines was multiple recordings of clicking the tongue layered together.

Often times when I need sounds, I will just turn on a mic and try to emulate the sound we need.

The trick here is to always *layer sounds*.

A single thump or click will sound awkward, but if you put five different ones together, it will work in your game.

We don't always resort to mouth recordings either. We've recorded sticks breaking, stomping on the ground and even revving an engine of a car.

Of course, if you know how to play a musical instrument that is always a bonus. You can record a few key notes for various aspects of your game.

Buying Pre-made Assets

This is the easiest and very popular approach to getting assets for your game. There are a multitude of sites that sell both art and sound files. Here are some of our favorites:

Art Asset Sites:

- Graphic River – Great art site with decent prices
- Shutterstock – If you buy a lot of art, they have a subscription plan.
- Fotolia – One of the largest sites out there for stock images
- iStockPhoto – Another large and notable stock image site
- Scirra Store – Graphic store for Construct
- Game Buffet – New site dedicated to game graphics
- Open Game Art – Site featuring free game art that is open source

- Atelier Store – Few pixel art items here sold in sets

Sound Asset Sites:

- Audio Jungle – High quality sound effects and music
- SoundSnap – Massive database of commercial sound effects and music uploaded by users
- FreeSound.org – Free sound database (not all offered/verified royalty free)
- Public Domain Sound – Public domain sound database (not all offered/verified royalty free)

Just be careful. You don't want to have a game that looks, and sounds, like everyone else's. Especially if you decide to get it published. Publishers hate seeing the same game art and sound over and over… and as you can imagine, they see *a lot* of games that never even get released.

Outsourcing Art & Sound

It's not the cheapest or quickest, but if you want a truly custom game, then this is the best option.

Price is extremely interesting with art. Different artist value the price of the work on vastly different planes and it's not always relative to quality. You can expect to pay anywhere from $200 to $2,000 for game art, so it's very important to get multiple quotes and shop around.

Sound is similar in the battle for quality and cost, but it's less widely varied and the cost is much cheaper.

As for the hiring and interview practices, the same rules that we outlined in the "Hiring A Programmer" section of this document apply. We've found the best outsource sites for design and sound are Upwork and Freelancer, though most of the others provide these services as well.

The final place you can look for artist and sound engineers who might be above the average quality seen on outsource sites is places where the elite hang out. Check out highly rated designers on sites like Dribbble and Behance. For sound engineers you can check forums like GearSlutz or FutureProducers.

SECTION 3

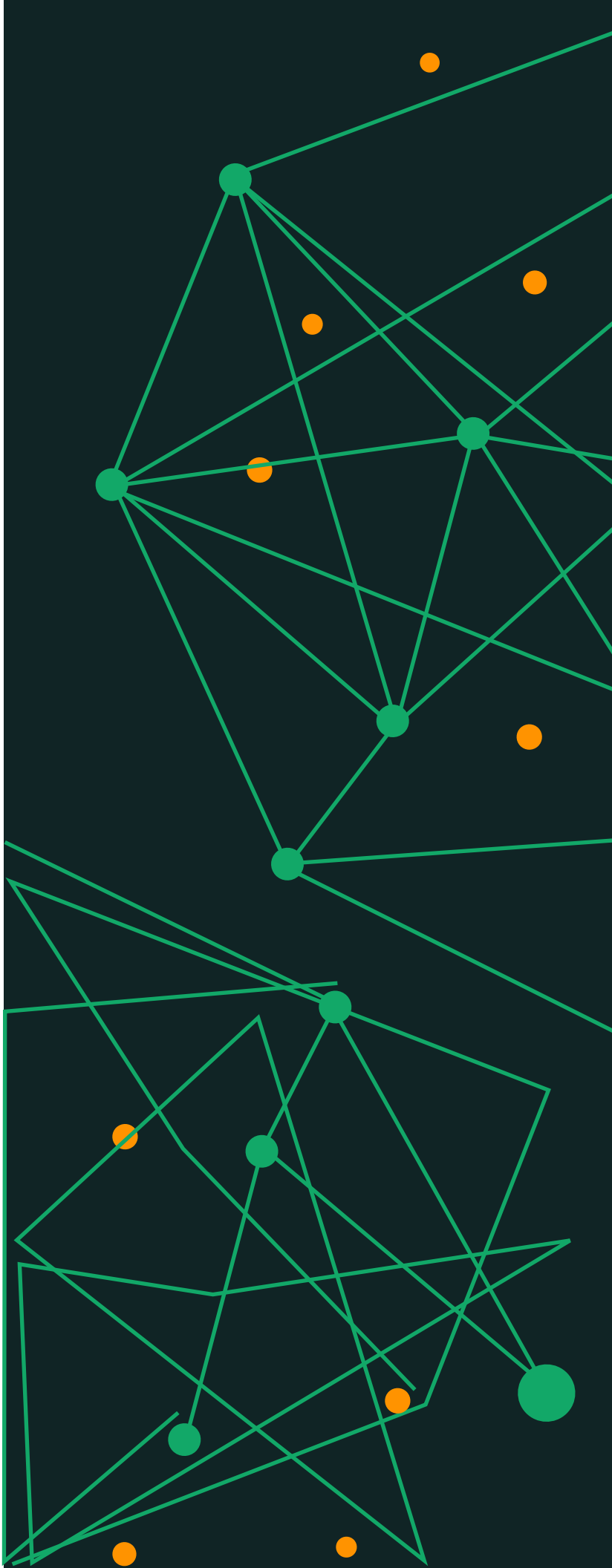
THE GAME

Game Design

In its most basic form, game design is simply the rules of your game.

These rules define the gameplay, the physics, the graphic design and ultimately the retention for your game. It's an all-encompassing term that describes the overall feeling and experience of your game.

In this section, we're going to focus on the most important aspects of game design: The game universe, difficulty growth, skill balancing and the overall experience.



Game Design: The Game Universe

The game universe is more than the rules of your game. It's the universal *laws* of your game that you create when building a new world.

For example, how heavy is the gravity in your world? How about the ground, is it bouncy? What color are pipes in your game? Does the sky change colors? If so, what hues?

All of these elements, and much more, should be consistent in your game in order to create the feeling of a living breathing universe.

Many times we test games from new hyper-casual game makers, and their universe is all over the place. It's important to create these laws early on and stick with them throughout the full creation of your game.

The concept of universal laws was explained to me by our CTO, Nik Rudenko. Nik comes from a background in 3D animation, and we were discussing the similarities between worlds in 3D animation and video games.

When Nik first explained this process to me, he used machinery exhaust as an example.

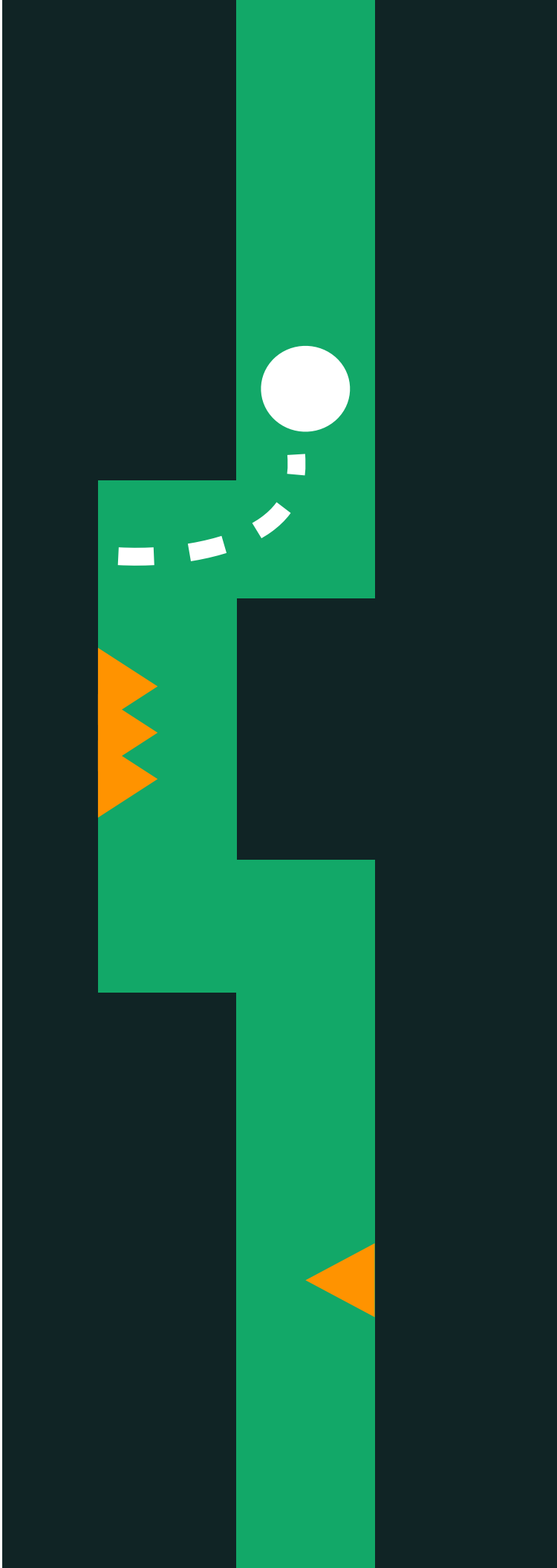
Let me explain:

Exhaust is something we see daily from cars, factories, industrial machines and more, but we never think how similar these various exhausts are in appearance. In our real universe, the exhaust is comprised of many different gasses, grey in color, lighter than air, and has a mass that quickly dissipates.

But what if in your game, you had a different energy source than burning fuel?

What if, instead, you used a new energy source made of lasers. Maybe, instead of grey exhaust, it would have small green beams coming out of pipes.
If this was a universal law, then you might want to add the same green beams for other sections of your game, like the factories in the background, or the exhaust from a train.

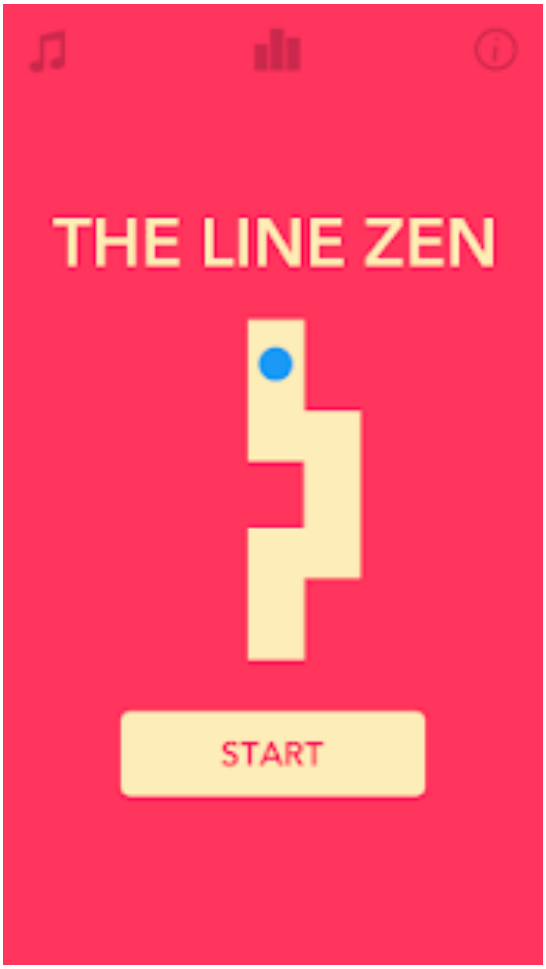
In Super Mario Bros, you know the physical and physics properties of the



pipe. You are aware that some of these pipes will lead to new places, some will have enemies, and some will be decoys.

You know that when Mario enters quicksand, he will slowly start to fall and be consumed by the danger. These universal laws of Super Mario Brothers quickly become ingrained in your mind and help you get lost in the world that *Shigeru Miyamoto* created.

Game Design: Difficulty Growth



Great games, like great stories, take you on an *adventure*.

It doesn't matter if it's a real adventure, like a *Legend of Zelda* game, or a feeling of adventure you might gather from the increasing difficulty in a more simple game like *The Line Zen*.

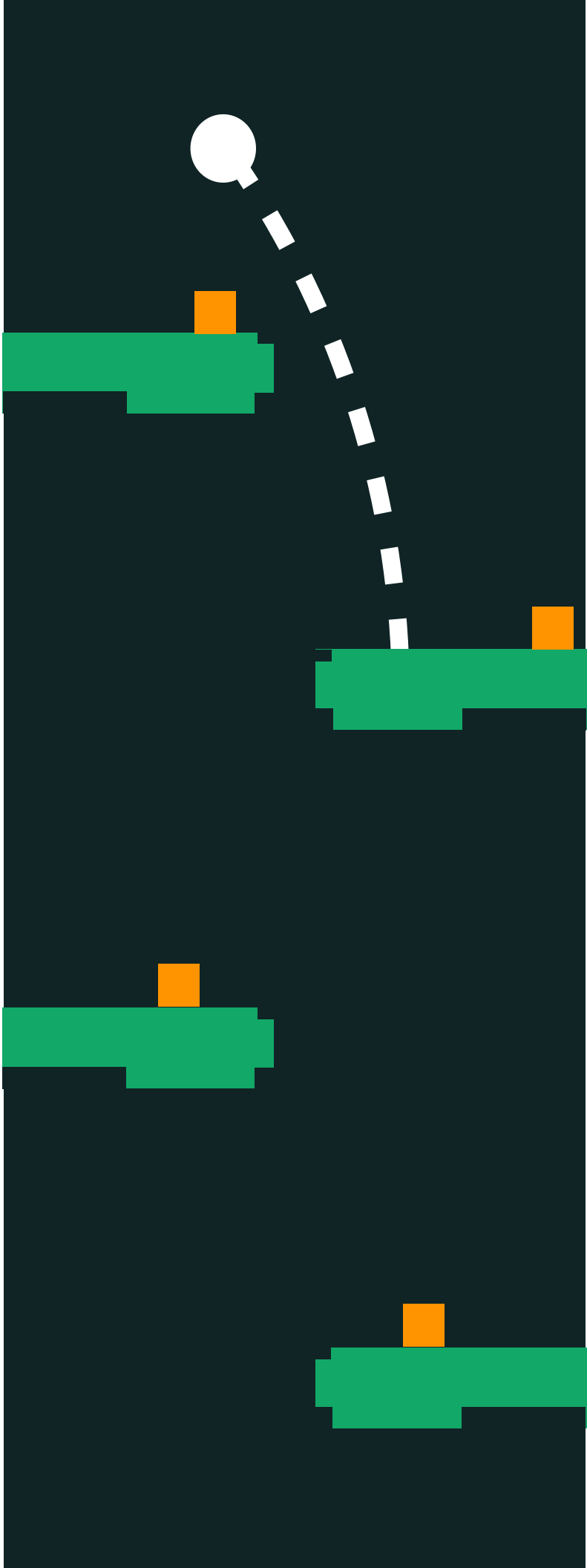
To create this feeling of adventure, you need to make sure your game has a subtle growth in difficulty. This subtle growth is usually established by the gameplay, or environment, changing over time.

For instance, in my game, *The Line Zen*, levels start becoming more involved after 10 seconds of play. This is a classic way of increasing difficulty in a video game. We keep the character the same, but the outside forces increase in challenge.

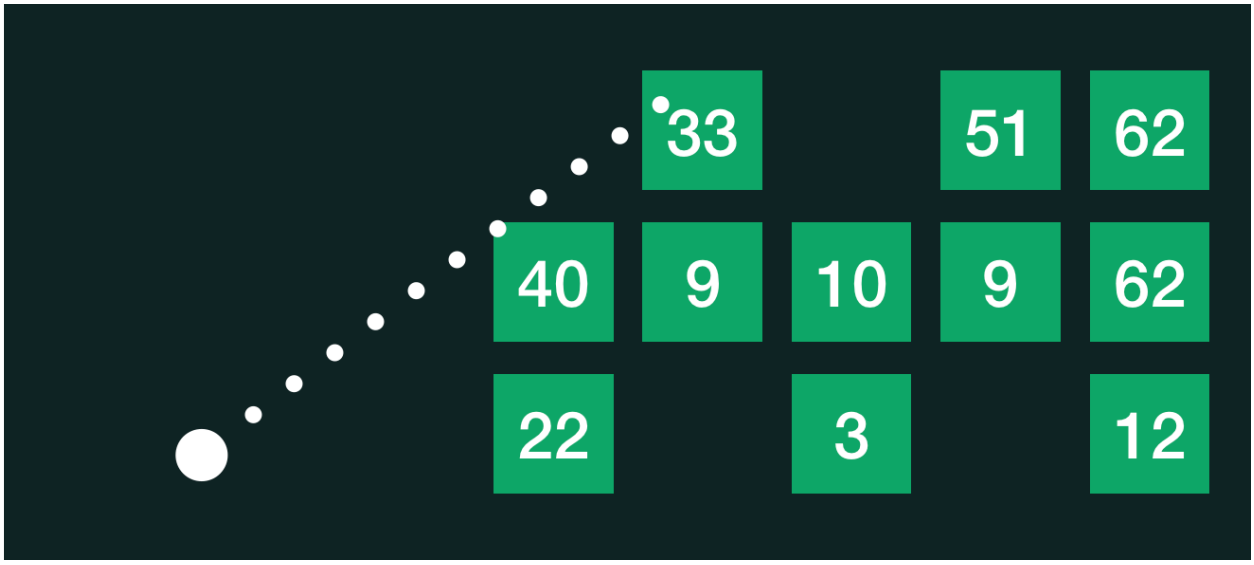
As we are now seeing more often, another way to increase difficulty is changing the environment around the player to make it harder, and then also the player itself to make it stronger.

This is a clear nod to classic role-playing games (RPG) where characters level up various skills to defeat more powerful enemies. We're now seeing these RPG elements being used in minimal mobile games like Ballz from Ketchapp.

Ballz grows in difficulty over time by the numbers inside the blocks increasing (i.e., health). You start off



with blocks in the low teens, but by the end of your session you are battling blocks in the 60s, 70s and higher.



To offset this increasing difficulty, you are also picking up more ammo (i.e., balls) that you can use to defeat the stronger blocks.

This extra layer of complexity is similar to leveling up an RPG character's damage to fight stronger enemies, and we will see more skills like this being upgraded in mobile games.



Game Design: Skill Balancing

While Difficulty Growth is necessary, it must be done with care. If your game becomes too difficult to fast, players will immediately exit.

This is where Skill Balancing enters into the equation.

Skill balancing is the art of making your game's difficulty enjoyable to a broad audience, while still keeping the expert players entertained. It's a balancing act that works hand-in-hand with Difficulty Growth.

Skillbalancing is the rate that your difficulty grows in your game, and unfortunately, most people get this wrong. We often see games that are too hard, and sometimes, though much more rarely, we see games that are too easy.

Regardless of the initial starting difficulty, we almost always see games that do not increase in difficulty at the perfect rate to keep users entertained for long durations.

The solution for this is two-fold. First, you should be playing and analyzing popular games that are similar to your game. See how long your average play session is and how long until you start improving.

Second, you should play test your game, on new people, a lot. Look for points of boredom, exhaustion, disinterest, and confusion, and then analyze, edit, and retest on a new subject.



Game Design: Overall Experience

The final part of game design is to focus on how your game is tied together. This is listed early on in this document, but it's something you should constantly be revisiting.
When we are making games, we're always asking questions like:

- What does this game feel like?
- How does this game flow?
- How do new users react?
- Do all the parts of the game design work together?
- How is the overall experience?

Before answering these questions, you need to set a benchmark, and an excellent way to do this is finding a reference. Find a game that is similar to yours and very successful, and think of how your game compares to it.

If your graphics aren't as high quality or the gameplay is not as refined, then ask yourself why. Find out what is missing in your game that makes the other one work so well.

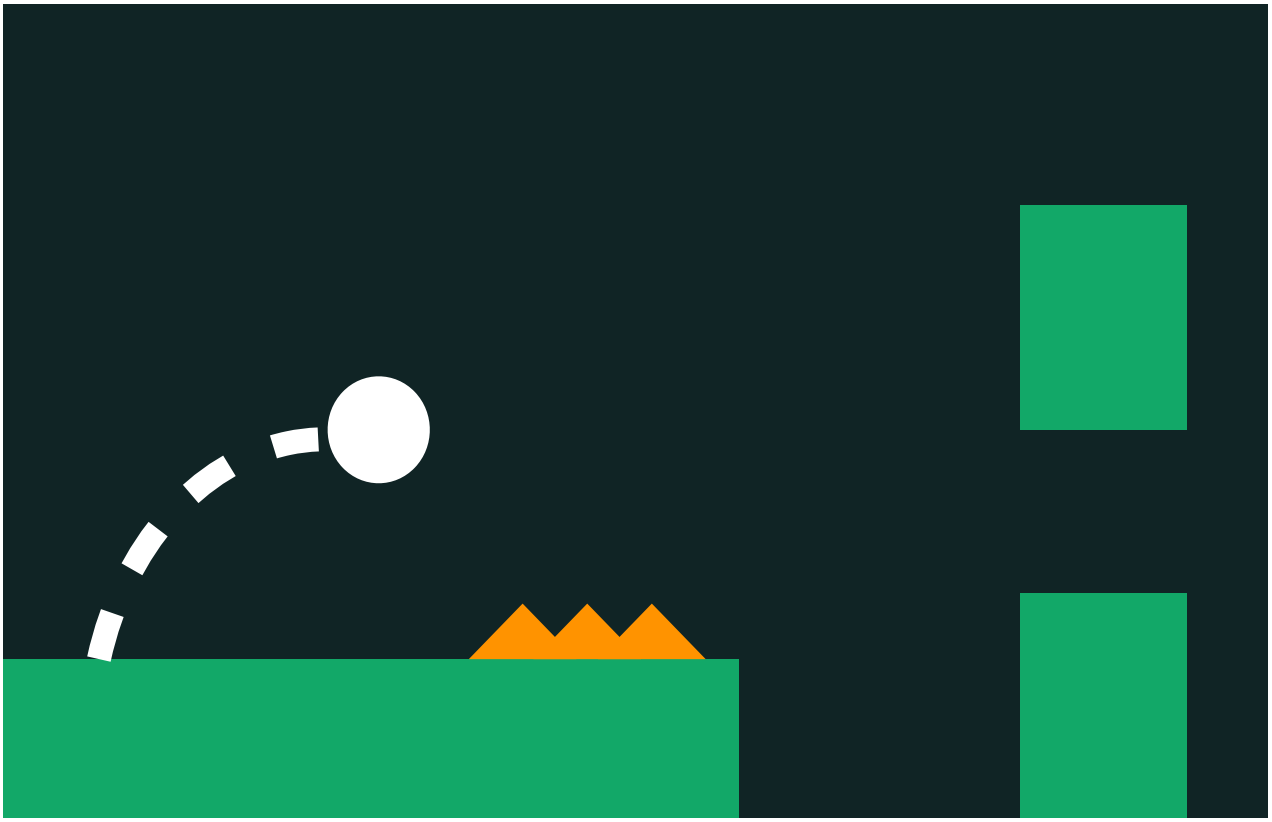
Graphic Design

Create the most amazing game that has ever been seen, and no one will care if it is ugly. It's an unfortunate truth, and the graphics of your game, especially in the hyper-casual market, constitute the barrier of entry with a publisher.

They will take one look at your screenshots or video, and immediately decide if it's worth pursuing.

It is your first impression and a *significant one*.

To nail the graphics, you'll need to focus on four key areas: consistency, object sizes, evolution and user interface.



Graphic Design: Consistency

Just like the universal laws previously discussed, your game's graphics should be consistent throughout.

Everything from the art style, to the characters, enemies, and style design should be consistent. Characters should be similar. Power-up icons should be similar. Even your *platforms* should be similar.

If the first one a user sees is pixelated with notches, then the future ones you see will follow these design principles.

For example, in Minecraft, all of the voxels have the same pixelated style, just with different patterns.

This consistency is key to making your game look great, and hyper-casual publishers won't sign a game that doesn't look amazing.

In the beginning, you should create a list of graphic design principles your game should follow throughout.

Graphic Design: Object Sizes

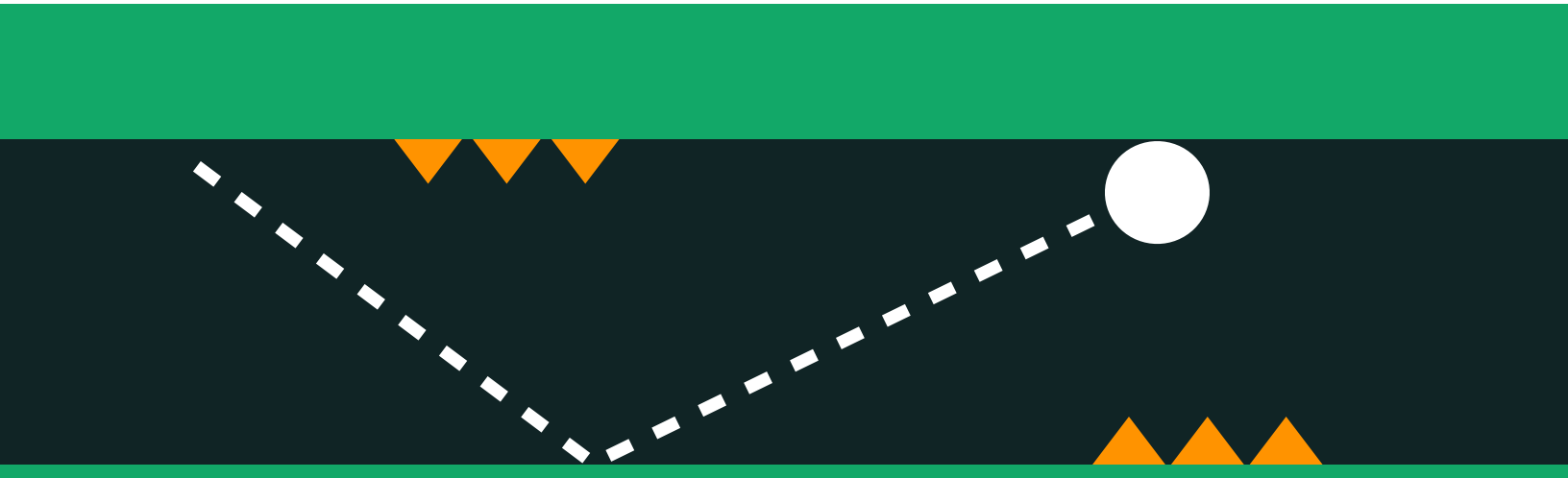
Object sizes should not only be consistent, but *typically smaller* than what most new developers plan for.

This has always been one of the biggest gripes we have when developers show us games and something I've been very vocal about in the past.

It seems most people forget how small the objects are in a game when they first play it. This is especially true on mobile devices, where characters are typically smaller than your pinky nail.

In fact, the most popular games of all time have small characters. A classic example is Super Mario Bros. Game designer, *Shigeru Miyamoto*, knew precisely what he was doing when he made the characters and objects small in his game.

When you use smaller objects, you're able to do more with your game. It's easier to create interesting game concepts because your game has so much open space.



Graphic Design: Evolution

The hyper-casual game market is constantly evolving, and it's important that your games evolve too.

Studying trends is one of the best ways to do this. They're reliable indicators of what's currently working and what's not. You don't want to end up wasting your time making a game that no one is going to like or be interested in playing.

So, make sure that your game design focuses on a current trend.

For example, 'minimal' is a trend in hyper-casual, but try to go beyond that. Within this subsection of graphic design, there have been multiple pattern changes. In the last two years, we've seen Flat Minimal then Isometric Minimal and now a movement back to Flat Minimal, but with more style (i.e., the recent influx of neon styled flat minimal design games). It's constantly evolving and changing, and it's something that game developers must keep an eye on.



Graphic Design: User Interfaces

A poor User Interface (UI) can ruin a game. It is *essential* to find a UI reference that you can use to help guide you along the way.

Try to look at the most popular button sizes and placements used in published games to get an idea of the best direction to go. Much like characters, people tend to make unimportant buttons, like social sharing, way too big.

Consistency is also key. If the restart button is green, but all other buttons are grey, then don’t change this around on just one screen. Stick to the same design principles throughout.

Finally, less is truly more. It's rare to see games with not enough UI buttons. Usually, it's the reverse problem.

Game creators love to clutter their UI with unnecessary buttons, but publishers and players hate it. Avoid making this rookie mistake and keep your user interface clutter-free.

Level Design

Level design is the process of actually building the levels, missions, or stages in your game.

When you're designing levels, the goal is to create a game environment that your players can explore and interact within a fun and challenging way.

We’re going to focus on the most important aspects of level design: uniqueness, difficulty, space, and inspiration.



Level Design: Going Unique

Level design is the one place that you always want to innovate. Try to add in new elements you've never seen before.

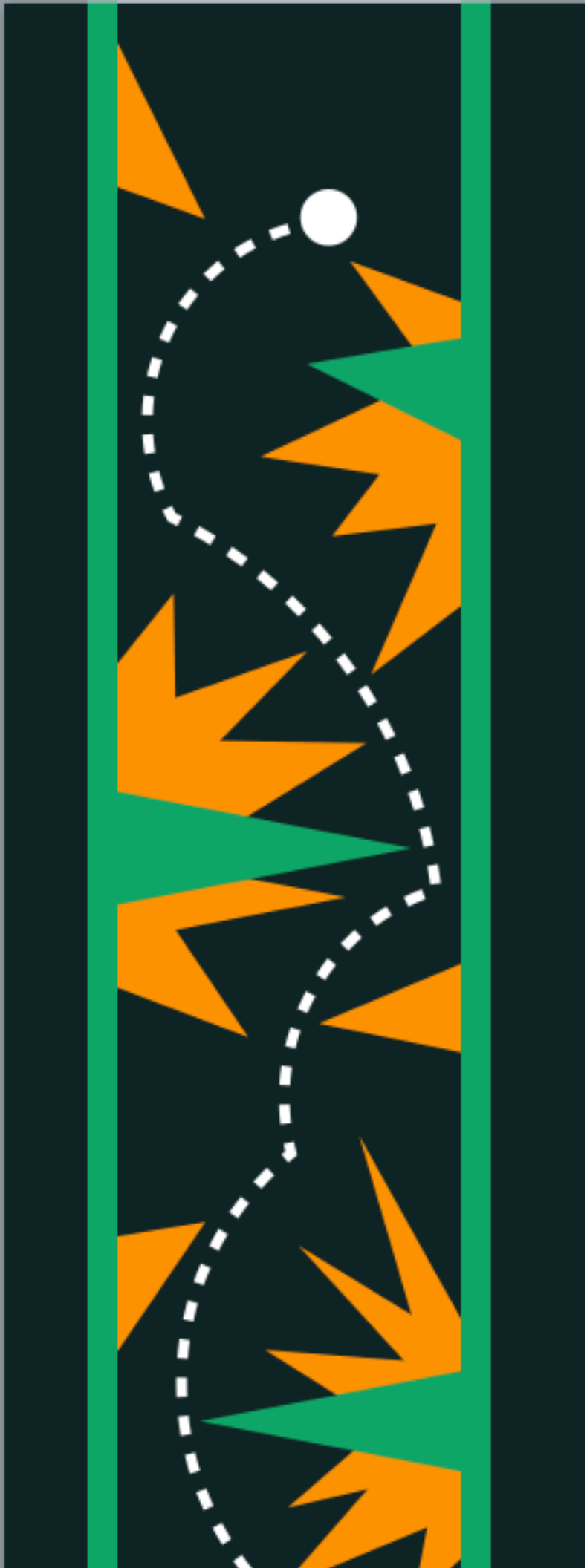
Do something *exciting*.

Surprise your players. Think of what excites you in games and explore those notions. Add trap doors, explosions, particles and more.

With level design, it's important to be unique and think outside of the box.

An excellent way to get inspired is to play indie games. Sign up for steam and play new indie titles. Check the indie games subreddit and start following #indiegames hashtags on Twitter and Instagram.

While you would *never* copy someone else's unique level design, there might be a way to twist it into something even more interesting for your game.



Level Design: Difficulty Thresholds

Most developers accidentally make their games extremely hard, and it makes sense. When making a game, you become an expert at your own game. You end up being the best player there ever was of your game, well, until its release.

Because your skill increases as you are making, and testing your game, this makes the level design difficulty harder to manage over time.

By the end of the development, new developers rarely have a gauge of what is too easy or too hard.

This is when testing on new people is instrumental.

Level Design: Space

Another common issue we see, and have made ourselves, is creating level design that is too tight.

You want *physical space* in your game. Let it *breathe*.

Make it challenging not by confining them, but because you have a genius level design. If you try to increase the challenge solely by restricting, you'll just create a frustrating experience. You'll turn your game into a tedious endeavor and cause players to resent you when they are defeated.

That should never happen in most games and *especially* hyper-casual mobile games that publishers like.

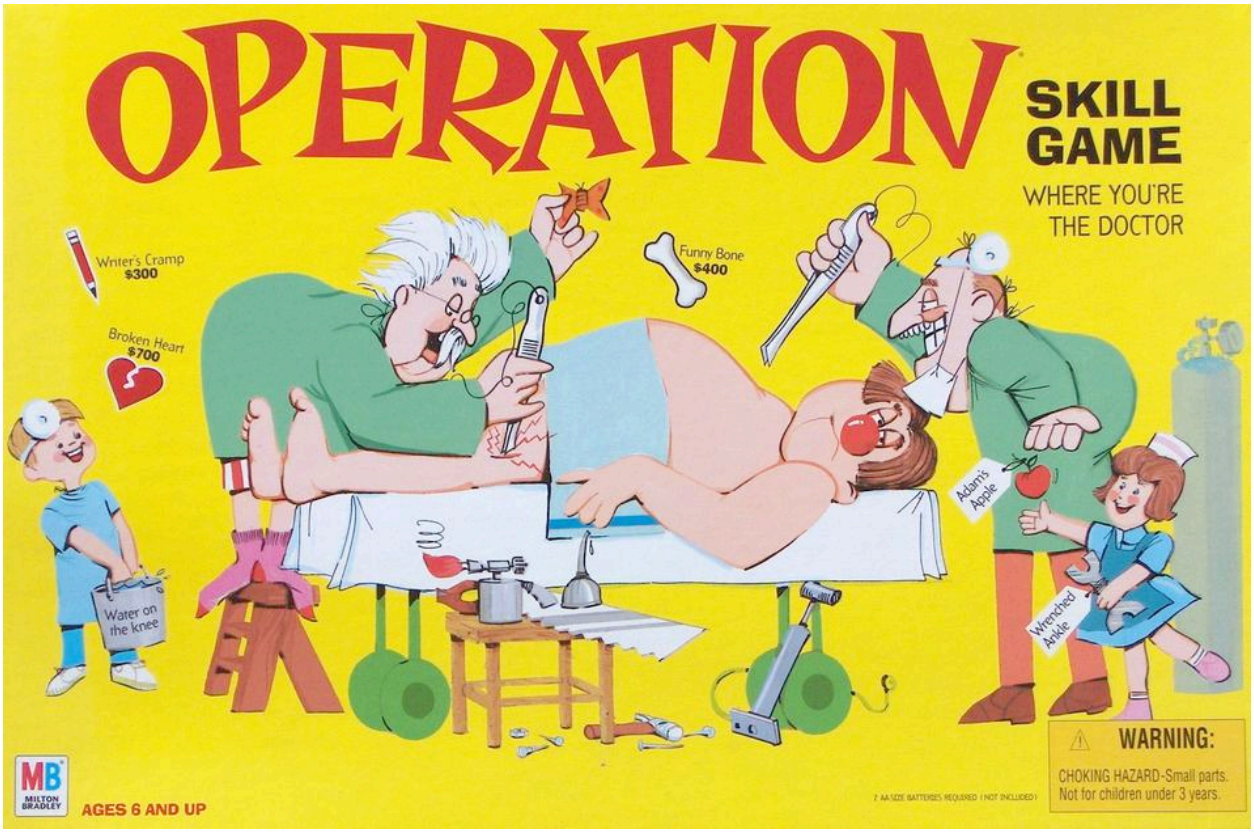


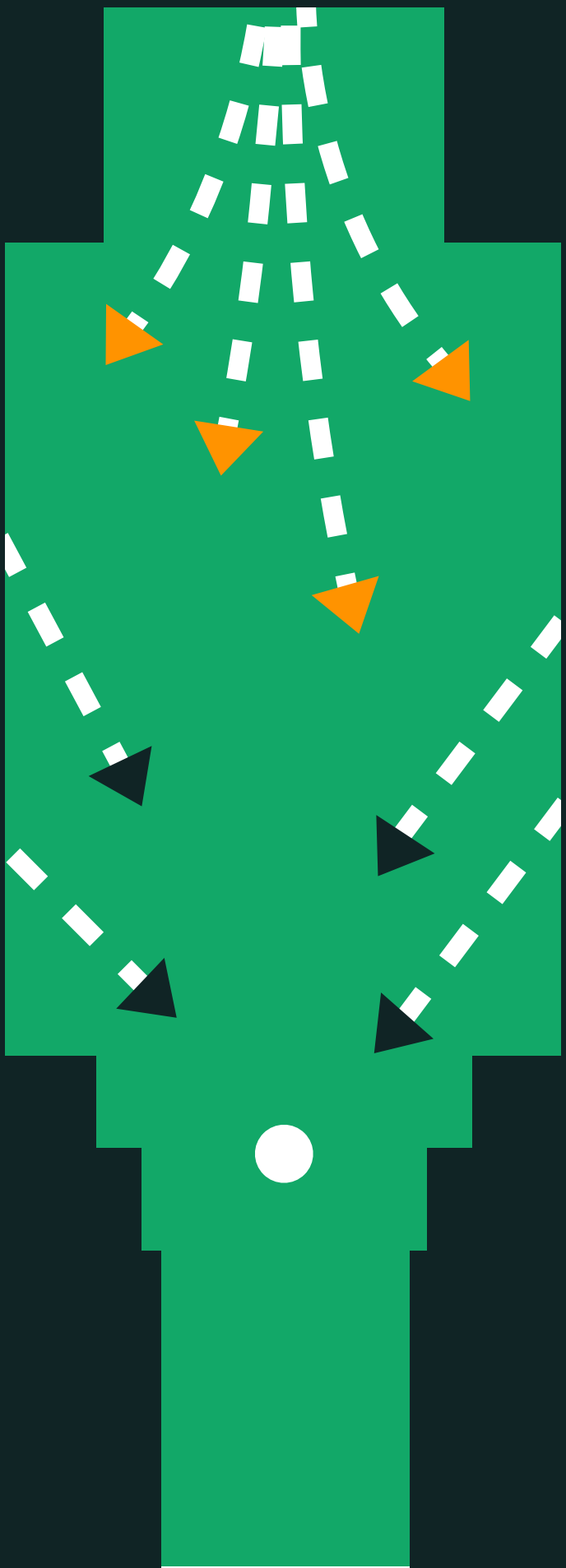
An important rule we use in our game company is this: When someone dies playing your game, they should feel that it is 100% *their fault*, and not the fault of your design.

If they feel like they were cheated, then your levels are too hard, or more specifically, too tedious.

That is even worse.

The first issue to check in this case is how much space you're giving them in your level design. Have the character be able to freely move around and don't force the player to feel like they're playing the classic board game, *Operation*, when playing your game.





Level Design: Inspiration

Model and Improve is the secret technique that the most successful game developers use. It's been practiced since the dawn of video games as a way to find inspiration.

The concept is simple:

Study popular but similar games and then brainstorm ways that you can improve the different aspects of it to make it even better.

Almost every hit game from Pong to Minecraft was inspired by something else.

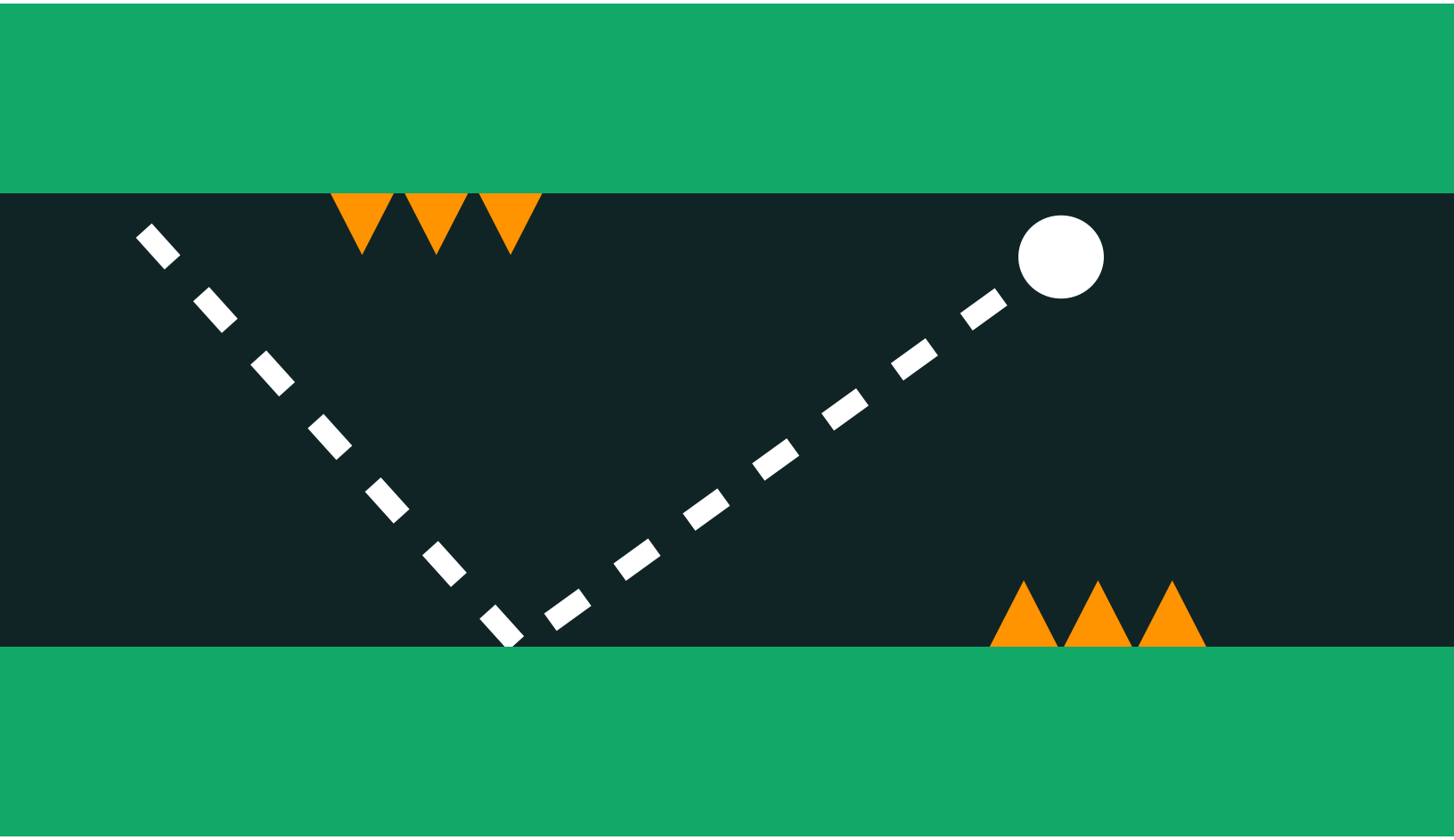
It's ok to get inspired, but don't forget to make it unique. That's the 'improve' part of the equation. Always make sure your game has something special, even if it's just a small tweak.

Don't just copy. Aim to make your *version* of the game different.

Retention

Retention is how often someone comes back to your game, and how long they play. It's become the most crucial analytics point for publishers because it directly results in viral growth, chart placement, and ultimately revenue.

More and more publishers are putting a strong focus on retention, and in this section, we'll highlight the four most important aspects of retention: Wonder, Surprise, Annoyance, and Competitiveness.



Retention: Wonder

Wonder is adding elements in your game that will excite players and have them curious to what's coming next. You can induce wonder by using innovative level design, perfectly developed skill balancing, new elements brought into the game, and end game content for your advanced players.

An excellent example of this is Super Mario Bros 3 where they introduced multiple new gameplay elements slowly as the game progressed, instead of all at once. Players would stumble upon a special block portal or gain a magical suit with the ability to fly or turn to stone to avoid approaching enemies.



This created wonder and kept users playing so they could see what would happen next.

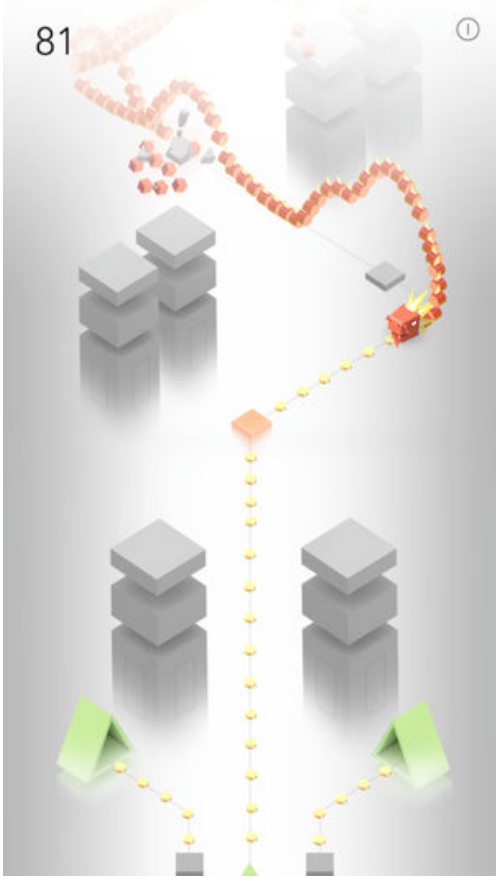
Retention: Surprise

Surprise is Wonder's exciting cousin. They work together to create a strong emotional reaction to your game.

While wonder keeps the users on the edge of their seat with excitement, surprise gives them the elements that keep them hooked. Sometimes the surprise is how well the wonder paid off, but usually, it's something that catches them off guard and is unrelated to the wonder.

For example, in Rayman, the first time you ride the bugs it's a huge surprise and something you did not see coming in the game.

With our game Endless Sky, we added crazy trails to new characters that you would not think we're going to be there when you first unlocked them.



Of course, this surprise immediately leads to wonder, because you're curious how the next trail will look.

Retention: Annoyance

Annoyance is a fascinating emotion. You would think it would kill retention, but while it can, that is not always the case. In fact, it can add to retention as games from every generation has shown us.

The original Nintendo games were notorious for being difficult. So much so, it coined the phrase, “Nintendo Hard.”

Mobile games have been known to be frustrating to no end, and some people love to proverbially torture themselves playing games like Dark Souls.

Why does this work, and how can you use it?

It works because it’s the perfect level of annoyance that makes the player work for his goal. After putting in some serious effort and skill, the payoff for the win is huge. It adds a serious emotional element to playing your game, which is always a big win.

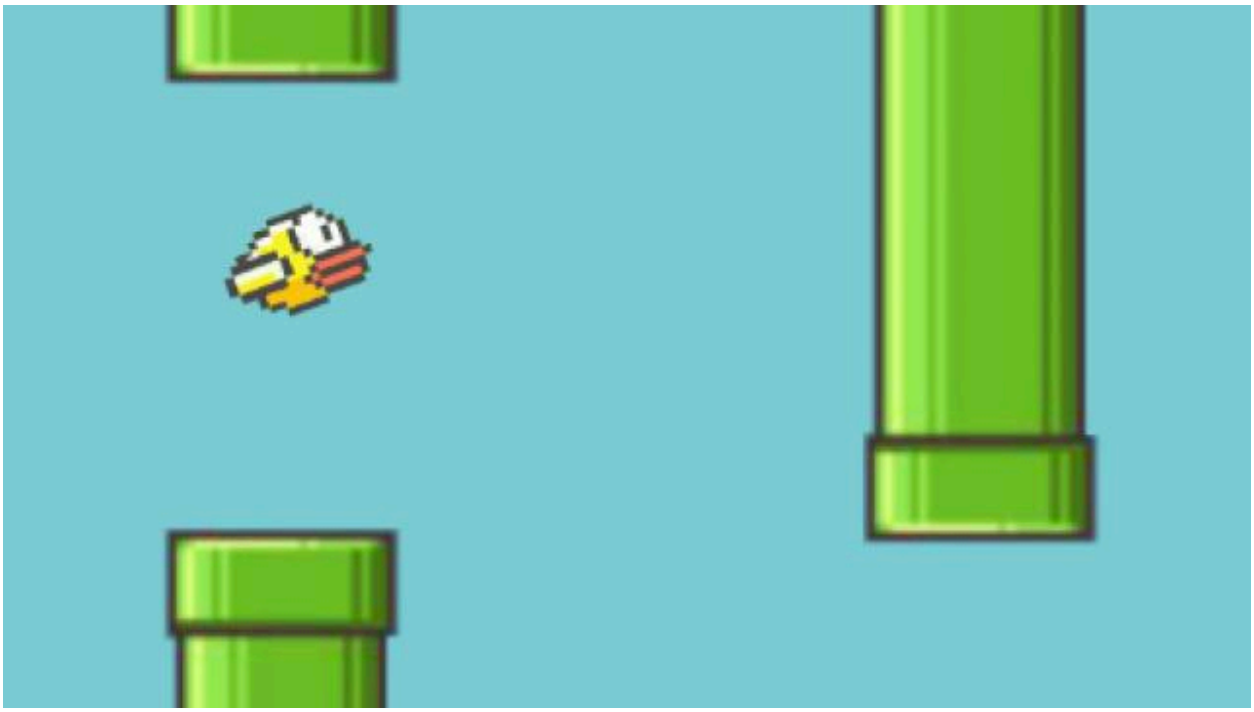
These types of emotions can also cause viral sharing of your game. People love to share about the harsh experience or conquest they’ve solved.

Of course, this can also backfire drastically and should only be attempted with extensive references and careful study.

Retention: Competitiveness

If there is a way for people to compete for high score, share that, and battle their friends, then you can have a huge win.

We first saw this take off on mobile with Flappy Bird. If you played Flappy Bird during its original debut, then you probably not only told your friends about the game, but challenged them to beat your high score.



Of course, this concept is not new and happened *decades before* Flappy Bird.

In the 1980s, back when arcades ruled the lives of teenagers across America, there were constant high score battles which still go on today.

We took note of this early, and most of our successful games grew vastly in popularity because of this viral sharing aspect.







SECTION 4

THE LAUNCH

Throw It Out The Window

The one thing I do not want to do in this book is waste your time, so I'm not going to talk to you about the "normal" stuff like getting press, posting on reddit or social media campaigns.

To be honest, that stuff doesn't work.

Maybe some of it is applicable if you're making an in-depth indie RPG game, or some other special scenario, but with hyper-casual that's really not the case.

It's a waste of time.

My advice? Do the same thing you did to make your game: Model and improve. See how other people are getting millions of downloads for their games, and just go do that.

Heck, we can even make it easier than that. I will just tell you specifically what we're doing, and what all of our friends who are crushing it are doing (and some of these guys, like David Riechelt who made Color Switch have over 100 million downloads).

What Not To Do

OK, now before we continue, let's talk about what not to do. These are some common, and not so common, mistakes that I have seen tons of people make (including myself many times over).

Here's my 10 Biggest App Launching Mistakes:

1. Don’t forget how important design is for your game

If your game does not have 10 out of 10 design, then it will fail. I'm dead serious. You can not get by on 9 out of 10.

It has to be up there with the best (even if it's a simple game).

We have a very simple rule for our games: It has to look BETTER than the games currently crushing it in the app store. If you are not a designer, and you don't have a good eye for design, then make sure to get lots of opinion on the look of your game.

2. Don’t focus on the web. It doesn't matter.

This is an interesting and potentially controversial mistake I see so many people make. Almost every app marketing list you'll find online will say "Don't forget to focus on your web presence!" and I could not disagree more with hyper-casual.

With the exception of partnering with social media stars, I've never seen someone create a hit hyper-casual game because of a web push. The truth is, the web is noisy and that's now how any of the top hyper-casual games are ranking high in the app store.

3. Don’t forget to push for an Apple Feature

An Apple Feature can get you 200,000+ downloads, even if you don't have a publisher. How do you get their attention? Well, for starters, if you have a great game you can pitch it to the feature team here: https://developer.apple.com//contact/app-store/promote/

4. Don’t ignore ASO (App Store Optimization)

Yes, I know at first this might seem like I am contradicting myself (as I do say in this very document ASO is dead), but what I mean is this: ASO will most likely not be an avenue that brings you in millions of downloads, but if you DO have a hit game it can definitely help keep you in the public eye longer.

Sites like Sensor Tower and App Annie have great information on App Store Keywords that work. We still research here, even though it's not our primary download driver anymore.

5. Don’t waste time on huge social media campaign (unless you are partnering with an influencer)

Listen, I understand this is another potentially controversial one.

Almost everyone who starts making mobile games immediately starts posting on Facebook/Twitter/Insta with hopes to build a following and rocket their game to the top.

Honestly, I've NEVER seen it work. We don't even have a Facebook Fan Page for our games or game company (just Buildbox and my personal). It just doesn't translate to downloads unless you have an existing ENORMOUS following like the Kardashians or Fortafy.

6. Your icon is your first impression

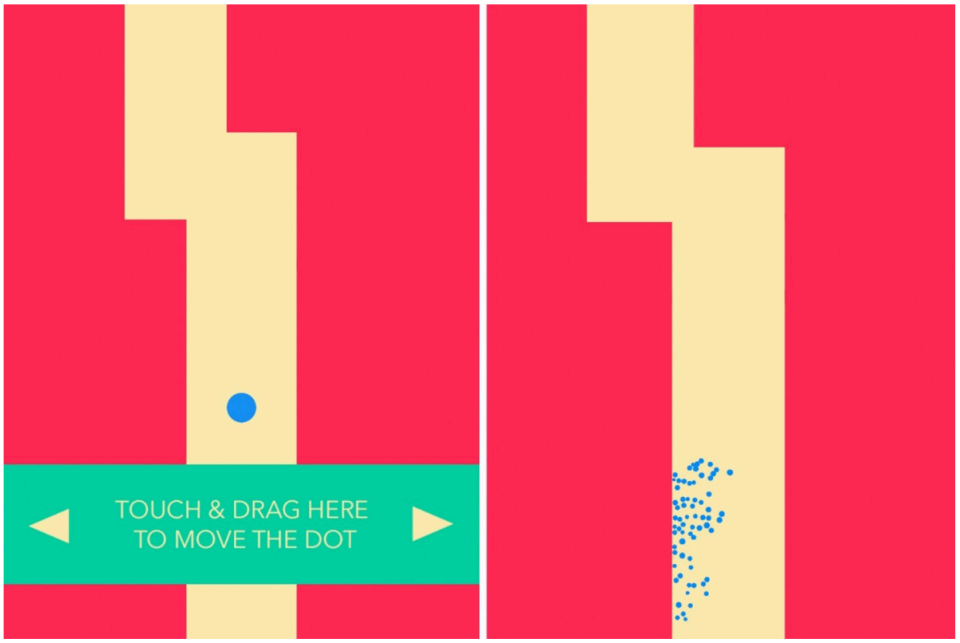
We've tested icons a lot, and it definitely matters. With hyper-casual, you want to try and show what the gameplay of your game is with the icon. If it's an avoidance game where you avoid spikes, then show a ball avoiding spikes in the icon.

This is the first impression, and it should give the user an idea of how to play your game immediately.

7. Don’t focus on PR

We've been in the news a a few times with our games. The Line Zen was featured on the Washington Post, we've been on the featured page of sites like App Advice, and we've had many friends also have features on Silicon Valley Blogs, Major News Sites and more.

A look at The Line Zen app



All you have to do is guide a blue dot down a winding path without hitting the red walls or other obstacles. If the ball hits the wall, it explodes and the game is over. (Ketchapp)

By Hayley Tsukayama May 8, 2015 Email the author

None of it had a lasting effect, so I would not recommend wasting too much time emailing journalist.

The app store is simply too large now for an article to get you a Top 25 ranking. That said, if your game goes VIRAL and you have nearly ALL the news sources talking about your app... then yes, that can make a huge difference.

The problem is, you can't realistically make that happen. That's more Flappy Bird syndrome that happens from a slow domino effect usually

sparked from something completely random. In Flappy Bird's case it started as a meme on Twitter where people would leave funny long reviews on Flappy Bird.

This affected the search algorithm at the time and pushed it higher in the ASO rankings. It became a phenomenon and then went viral on the news. Typically mass news happens as a RESULT of viral growth, it does not actually cause it.

8. Never pay a website for promotion

Let's start with the basics. When you start researching how to launch an app, you'll indubitably come across a website saying they will promote your app for a semi-large amount of money. Do not take them up on this. It's not going to make a difference.

9. Don’t buy traffic (unless you know the metrics)

Another big one. I've seen a lot of people go in and attempt to buy Facebook/Google/Etc traffic to their game thinking they will hit the top charts.

They learn after spending thousands that it's not an easy task. You have to hit really strong numbers to make this work. Specifically you want a Cost Per Install of $.40 or lower, and retention of at least 25% after Day 7. Those are hard numbers to hit.

10. Actually, don't spend any money on your first game

My first game sucked. Your first game will probably suck too. A lot of people get in this business - think they have something hot - and go blow a bunch of cash promoting their first game. Please don't do this. Use your first game as a learning experience. Keep the hopes in check and go for the journey. You're second game will be 5X better.

What Not to Do (Times Ten)

This one gets its own section aside from the top ten biggest mistakes because this needs to be emphasized. I cannot stress the importance of this enough.

Also, since we're focused solely on hyper-casual, the ones below are important ones I don't see people mention often:

11. Never make a game a hyper-casual game in landscape mode

With hyper-casual games, it's all about pick-up-and-play while you are in line at a restaurant, meeting up with a friend, or even waiting on a movie to start. These play sessions are not long, and more importantly... they happen quickly.

We've tested this like crazy, and believe it or not, landscape games do not convert near as well as portrait games solely because you have to turn the

device. This second long rotation is enough to detract people from using the game and causes a huge dip in retention.

This is why Phases was our first, and last, landscape hyper-casual game.

12. Always make hyper-casual games either one-touch or swipe

There is a good reason why all hyper-casual games have extremely simple controls, and it's for the same reason they are in portrait mode. When you have simple controls, people immediately know how to play your game and this vastly increase retention.

One-touch games can played by anyone, regardless of age or demographics. Swiping games while not quite as wide, are very, very close, as we saw with casual games like Angry birds, The Line Zen, Temple Run, and even drawing games like Draw Something. If you take a look at hyper-casual games, you'll notice they never have on screen buttons or complex controls.

The Key To Any Business

To create any successful business you need two things: *traffic and conversions*.

This is true regardless if you’re a SAAS (Software as a Service) company like Buildbox, a car company like Tesla, or if you make mobile games.

In theory, conversions are pretty simple. You have to have a great product and some system to convert that product into dollars. With mobile games, this has always been either a premium or freemium business model.

With premium you charge the money up front, and with freemium you make money with ads and in app purchases. Pretty simple, but the traffic side of the business? Not simple at all.

When I started in late 2010, there wasn't a lot of competition in the app store. My first game, called Kolo's Journey (which was OK, but not amazing), got featured by Apple.



Back then if you got featured on Touch Arcade, you would be in the Top 100 games. You would also probably get featured by Apple.

That might seem like the "golden years", but that's not really true. While it was easier to rank, it was also a LOT less people on mobile devices. One year ago Apple announced it has sold over 1.2 billion iphones.

That is *insane*.

To put that in perspective, in 2010 when I started there were *only 42 million iPhones sold*. That is 3.5% of the phones sold today, so in turn, that's 3.5% of the revenue you could make in the app store back then.

So, was it easier to rank in 2010 than 2016? Yes, but your app was see by a small fraction of the people compared to today. Easier launches, much less top side.

This is one reason we've generated more revenue in the past 2 years than we have the previous 4 years. The other reason is, the traffic model…

A Game of Cat and Mouse

Now let's discuss traffic, because this is the real lifeblood of any business, and it's always been a bit weird when it comes to apps.

There has often been some sort of short lived "*secret*" that people would figure out and run as hard as possible until it quit working, and typically this worked around ASO.

Let me give you some examples:

In 2010, people got traffic from Apple features or outside sources like press mentions or email lists. Again, it was easy to rank then, but less upside because of so few people on mobile devices.

In 2012, people started figuring out App Store Optimization (ASO). If you're not familiar, ASO is simply showing up in search results in the app store.

Tons of tricks were built on the back of ASO until Apple and Google caught on.

For example, we found out that for a short while Apple was ranking the developer's *company name* higher than anything else. If you made a company called "Best Game, Inc", then you would rank #1 for the extremely competitive keyword "best game".

As you can imagine, Apple killed that pretty quick.

We then found out if you named your app a *single word name* with a period after it, then you would rank #1 for that term!

I had a game called "Games." and another one called "Music." and ranked #1 for both terms. That worked and I decided how far I could push it. I created games named "Free." and "Free Games., and yes, we ranked#1 for both terms as well, but only for a *very short* while. Apple caught on and stopped completely.

And believe it or not, this is just the beginning…

There were about one dozen stories like this I could tell. For years it was a game of *cat and mouse*.

Apple was constantly trying to shut down the loopholes it created, and finally, they nailed it down *tight*. They took huge strides in how search works on the App Store and closed every loophole you could think of.

The Death of ASO

Starting in early 2016 the tricks completely stopped working, and ASO died with it. Sure it's smart to choose keywords that effectively describe your app (i.e. words people will search for), but gone are the days where keywords will get you *millions of downloads*.

Apple changed their algorithm to primarily show apps that are already popular at the top of search results, which makes sense. That gives the end-user a better experience as they're going to probably play better games.

And that is ok…

Because through ALL the cat and mouse games, we got about 10,000,000 downloads. Since then, we've got 60,000,000 more downloads doing it the *right way*.

It's been 10X easier, and more fun, and we've made more money.

Pretty good combo.

The New Era

This is a new era, and as typical in these situations, a few other options have risen up to take place of the falling model.

Yes, App Store Optimization is dead. It's a crowded market that now gives the games on top even more leverage, but this does not mean it's the end for the hyper-casual game creator.

In fact, it's the opposite.

I'm happy to say our business is growing faster than ever and the new model that has replaced ASO for us is here to stay. It's actually consistent, which App Store Optimization never was for us.

Before I explain exactly what we're doing, let me show you every option that is currently working for people right now.

What Works Now

There are a few things that work right now in the mobile market for hyper-casual, and really any other type of mobile game.

Some are harder than others, but I'll list them all out below, then explain what we're doing (as well as our most successful customers).

Social Influencers

We've seen a lot of people doing well by either hiring or partnering with social influencers. Obviously there are the really big deals done with Kim Kardashian, but this has also worked out on a smaller scale.

We've seen people partnering with Youtube stars, Instagram famous people and more.

It's a good approach if you have a big following, are great at networking or don't mind spending some money to promote your game.

App Store Feature

We used to get featured a lot, but it's now been years since we entered the game business. For us, it's a bit irrelevant now. This is a tough one to base your business on, but I do know *one person*, who is not a publisher, who does it successfully. He literally has built a business solely based on getting featured.

The key here is to first meet someone from the editorial or business development team, and then make sure to add as many new features to your game as possible, and create polished products.

Even then though, it's a bit of a *lottery*.

If you can somehow make yourself desirable – and have Apple or Google chasing after you – then you have a real strong shot at getting lots of features… but again, easier said than done.

Buying Installs

Buying installs is a huge market dominated by big players like MZ, makers of Game of War, and King, makers of Candy Crush.



Because this upper echelon of developer has nearly unlimited financial resources, it's made purchasing traffic extremely expensive for game developers.

Some publishers have figured out buying ads to push games to the top (we'll talk about that in a second), but it takes two things to do it right:

1. You have to REALLY know the numbers before you start spending (i.e. the correct metrics across the board)

2. You need access to a lot of capital if it's going to work

The reason is, you'll not get profitable until a game is pushed into the Top 10 and you start getting residual downloads from chart watchers downloading your game.

To get to this level can cost upwards of 6 figures a day, so it's definitely something only reserved for people with experience in the arena.

Getting Published

The final option you have to get traffic is also the most consistent one.

The only other option that might match it in consistency is partnering with a social influencer, but that largely depends on the outside success of the influencer themselves overtime.



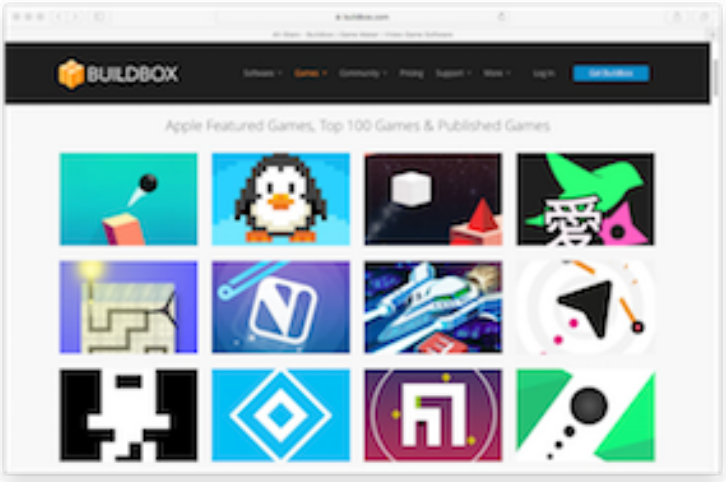
Regardless, probably at no surprise to anyone, getting published is our option of choice to get traffic to our games. The reason we pick this over other models is because:

- **Diversification** – You're not locked down to one single source (Apple Feature) or the monetary value of a market (buying installs). Just like every other major media market, there are many publishers to choose from and the list is only growing.

- **Creation Guidance** – We create games by analyzing what is working for publishers. This vastly helps us decide what to create, and ensures that we have at least a decent shot of the game doing well (we don't always hit home runs, but we almost always make it on base). If you get featured, partner with a celebrity, or buy installs, you are pretty much on your own. There is no roadmap to follow or guidance on what you should make.

- **Ease of use** – Not having to worry at all about the marketing has been a huge stress relief on our company. I used to stay up all night trying to figure out the latest ASO hack when we would get delisted. Now we just focus on one thing: Making good games.

Most importantly, it's been the most consistent way to grow a game business that I have ever seen. This is how every Buildbox customer who is crushing it has dominated the App Store. In fact, 70% of the games on our Buildbox All Star list were published.



Ok guys and gals, I hope you enjoyed this book.

I had a blast writing it, and hopefully I answered most of your questions on making and launching hyper-casual games.

Happy building and take care!

Trey Smith

BONUS

WEBINAR TRAINING

Want To Know How To Get Published?

As mentioned previously, I've helped a *lot* of people get publishing deals. I actually personally email publishers to connect them with developers who make good games.

So far, I've assisted in getting more than 30 hyper-casual games signed with top tier publishers like Ketchapp, Gameloft, Voodoo, Nanovation, Appsolute, Umbrella and more.

Unfortunately, I can only take on limited clients like this, but I have boiled everything down into an **hour long training webinar** that you can watch *for free* here:

Crush The App Store Webinar

This is my advanced training and it goes way beyond this book. On this event you'll learn everything about getting hyper-casual games published.

I'll show you things like:

- Live researching the app store and **coming up with ideas**
- **Screenshots of email conversations** Ii've had publishers
- Advice on **which publishers to go for when starting out**

- Exactly **how to pitch a publishers**
- **The three things** you never say to publishers
- Our **exact systems we use in my game company** for every aspect of getting games published (I break the whole thing down into sections)
- **Weird secrets** to what friends of ours are doing to crush it *right now*
- How to have hyper-casual publishers **coming after you** and more

It's called the **Crush The App Store Event** and it's completely free, so make sure to check it out:

Crush The App Store Webinar